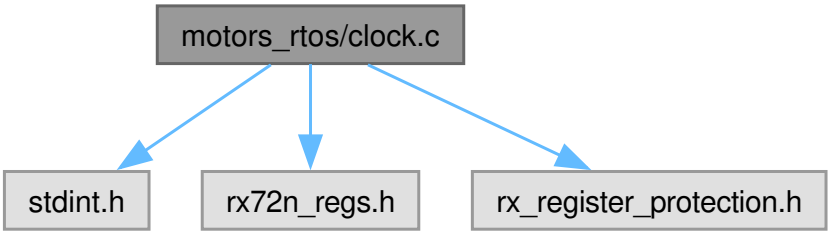
>>

>>

>>

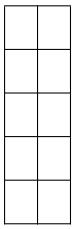
```
#include <stdint.h>
#include "rx72n_regs.h"
#include "rx_register_protection.h"
```



>>

>>>

enum clock_byte_constants_t : uint8_t



```
00059             : uint8_t {
00060   k_sub_clock_stopped = 0x01U, /**< SOSCCR: stop sub-clock oscillator */
00061   k_main_osc_enabled  = 0x00U, /**< MOSCCR: 0 = MOSC running */
00062   k_pll_enabled       = 0x00U, /**< PLLCR2: 0 = PLL running */
00063   k_oscovfsr_moovf    = 0x01U, /**< OSCOVFSR bit 0: MOSC stable */
00064   k_oscovfsr_plovf    = 0x04U, /**< OSCOVFSR bit 2: PLL stable */
00065 } clock_byte_constants_t;
```

enum clock_extra_addrs_t : uintptr_t

```
00028             : uintptr_t {
00029   k_packcr_addr = 0x00080044U, /**< Peripheral clock source: 0=PLL */
00030 } clock_extra_addrs_t;
```

enum clock_sckcr_t : uint32_t

```
00050             : uint32_t {
00051   /*
00052    * SCKCR dividers (ICLK half-speed vs production):
00053    *   FCK=/4 (60), ICK=/2 (120), PSTOP0=1, PSTOP1=1, BCK=/4 (60),
00054    *   PCKA=/2 (120), PCKB=/4 (60), PCKC=/2 (120), PCKD=/2 (120).
00055    */
00056   k_sckcr_value = 0x21C21211U,
00057 } clock_sckcr_t;
```

enum clock_timeout_t : uint32_t

```
00067             : uint32_t {
00068   /* Bounded poll counts (NASA Rule 2). We poll the hardware ready flag
00069    * (OSCOVFSR.MOOVF / .PLOVF) so the actual wait is hardware-determined --
00070    * roughly 10 ms physical time regardless of CPU clock. The upper bound
00071    * just prevents an infinite spin if the crystal is missing. ~500k iters
00072    * at LOCO 240 kHz is ~3 s, which is still much shorter than the prior
00073    * fixed 10-second busy-wait. */
00074   k_main_osc_poll_max  = 500000U,
00075   k_pll_lock_poll_max  = 500000U,
00076 } clock_timeout_t;
```

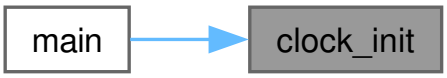
enum clock_word_constants_t : uint16_t

```
00035             : uint16_t {
00036   /*
00037    * PLLCR layout:
00038    *   bits[1:0]  PLIDIV    : 00 = /1
00039    *   bit [4]    PLLSRCSEL : 0 = MOSC, 1 = HOCO
00040    *   bits[13:8] STC       : multiplier = (STC + 1) / 2
00041    *
00042    * MOSC 24 MHz / 1 * 10 = 240 MHz. STC = (10 * 2) - 1 = 19 = 0x13.
00043    * PLLSRCSEL = 0 (MOSC) -> bit 4 clear.
00044    *
00045    * Production rx_clock_power_init.c uses k_pll_multiplier _div  = 0x1300.
00046    */
00047   k_pllcr_mosc_x10 = 0x1300U,
00048 } clock_word_constants_t;
```

void clock_init (
                void)

```
00082 {
00083   volatile rx_system_regs_t *sys = system_regs();
00084
00085   /* Unlock PRC0 (clock) + PRC1 (module stop) write protection. */
00086   *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00087
00088   /* Stop sub-clock oscillator -- not used on STAR. */
00089   sys->sosccr = k_sub_clock_stopped;
00090
00091   /* Start MOSC (24 MHz external crystal). */
00092   sys->mosccr = k_main_osc_enabled;
00093
00094   /* Poll the hardware MOSC-stable flag. Exits as soon as the crystal has
00095    * settled (typically ~10 ms physical time) instead of waiting a fixed
00096    * 2.4M cycles, which on the LOCO 240 kHz boot clock is ~10 seconds. */
00097   for (volatile uint32_t i = 0; i < k_main_osc_poll_max; i++) {
00098     if ((sys->oscovfsr & k_oscovfsr_moovf) != 0U) {
00099       break;
00100     }
00101   }
00102
00103   /* Configure and enable PLL: 24 MHz x 10 / 1 = 240 MHz. */
00104   sys->pllcr  = k_pllcr_mosc_x10;
00105   sys->pllcr2 = k_pll_enabled;
00106
00107   /* Wait for PLL lock (bounded). If we time out, the crystal is missing
00108    * or not driving -- DO NOT switch SCKCR3 to PLL or the chip will halt
00109    * on a dead clock. Stay on LOCO so the LEDs still respond and the user
00110    * can see we got past clock_init. */
00111   bool pll_locked = false;
00112   for (volatile uint32_t i = 0; i < k_pll_lock_poll_max; i++) {
00113     if ((sys->oscovfsr & k_oscovfsr_plovf) != 0U) {
00114       pll_locked = true;
00115       break;
00116     }
00117   }
00118
00119   if (pll_locked) {
00120     /* MANDATORY: raise flash wait state BEFORE switching to >120 MHz. */
00121     *memwait_reg() = k_memwait_one_wait;
00122
00123     /* Apply system clock dividers. */
00124     sys->sckcr = k_sckcr_value;
00125
00126     /* Route UCLK from main PLL (UPLLSEL=0). */
00127     *((volatile uint16_t *)k_packcr_addr) = 0x0000U;
00128
00129     /* Switch system clock source to PLL. */
00130     sys->sckcr3 = k_sckcr3_cksel_pll;
00131   }
00132
00133   /* Re-lock PRCR. */
```

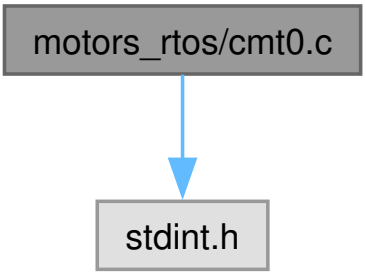
```
00134   *prcr_reg() = k_rx_prcr_lock;
00135 }
```




```
00001 /**
00002  * @file clock.c
00003  * @brief RX72N clock init for motors_rtos: MOSC 24 MHz crystal -> PLL 240 MHz
00004  *        -> ICLK=120 / PCKA=120 / PCKB=60 / FCLK=60 MHz.
00005  *
00006  * @details
00007  * Self-contained reduced version of src/rx_clock_power_init.c, stripped of
00008  * the logging, asserts, simulator branches, and PPLL/USB clock paths
00009  * motors_rtos does not need. Configures only the main PLL.
00010  *
00011  * Path: EXTAL 24 MHz -> MOSC -> PLL (x10 / 1) = 240 MHz -> SCKCR=0x21C21211.
00012  * After this routine returns, ICLK is 120 MHz (half of production's 240 MHz)
00013  * and PCLKA (the GPTW source) is 120 MHz, matching k_pclka_hz in
00014  * libs/rx_hal/inc/rx72n_clock.h.
00015  *
00016  * SPDX-License-Identifier: MIT
00017  * @copyright Copyright (c) 2026 Locked Inc.
00018  */
00019
00020 #include <stdint.h>
00021
00022 #include "rx72n_regs.h"
00023 #include "rx_register_protection.h"
00024
00025 /*
      ============================================================================
00026  * Register addresses not exposed by rx_system_regs_t
00027  *
      ============================================================================
      */
00028 typedef enum : uintptr_t {
00029   k_packcr_addr = 0x00080044U, /**< Peripheral clock source: 0=PLL */
00030 } clock_extra_addrs_t;
00031
00032 /*
      ============================================================================
00033  * Bit / value constants (mirrors src/rx_clock_power_init.c)
00034  *
      ============================================================================
      */
00035 typedef enum : uint16_t {
00036   /*
00037    * PLLCR layout:
00038    *   bits[1:0]  PLIDIV    : 00 = /1
00039    *   bit [4]    PLLSRCSEL : 0 = MOSC, 1 = HOCO
00040    *   bits[13:8] STC       : multiplier = (STC + 1) / 2
00041    *
00042    * MOSC 24 MHz / 1 * 10 = 240 MHz. STC = (10 * 2) - 1 = 19 = 0x13.
00043    * PLLSRCSEL = 0 (MOSC) -> bit 4 clear.
00044    *
00045    * Production rx_clock_power_init.c uses k_pll_multiplier_div  = 0x1300.
00046    */
00047   k_pllcr_mosc_x10 = 0x1300U,
00048 } clock_word_constants_t;
00049
00050 typedef enum : uint32_t {
00051   /*
00052    * SCKCR dividers (ICLK half-speed vs production):
00053    *   FCK=/4 (60), ICK=/2 (120), PSTOP0=1, PSTOP1=1, BCK=/4 (60),
00054    *   PCKA=/2 (120), PCKB=/4 (60), PCKC=/2 (120), PCKD=/2 (120).
00055    */
00056   k_sckcr_value = 0x21C21211U,
00057 } clock_sckcr_t;
00058
00059 typedef enum : uint8_t {
```

```
00060   k_sub_clock_stopped = 0x01U, /**< SOSCCR: stop sub-clock oscillator */
00061   k_main_osc_enabled  = 0x00U, /**< MOSCCR: 0 = MOSC running */
00062   k_pll_enabled       = 0x00U, /**< PLLCR2: 0 = PLL running */
00063   k_oscovfsr_moovf    = 0x01U, /**< OSCOVFSR bit 0: MOSC stable */
00064   k_oscovfsr_plovf    = 0x04U, /**< OSCOVFSR bit 2: PLL stable */
00065 } clock_byte_constants_t;
00066
00067 typedef enum : uint32_t {
00068   /* Bounded poll counts (NASA Rule 2). We poll the hardware ready flag
00069    * (OSCOVFSR.MOOVF / .PLOVF) so the actual wait is hardware-determined --
00070    * roughly 10 ms physical time regardless of CPU clock. The upper bound
00071    * just prevents an infinite spin if the crystal is missing. ~500k iters
00072    * at LOCO 240 kHz is ~3 s, which is still much shorter than the prior
00073    * fixed 10-second busy-wait. */
00074   k_main_osc_poll_max  = 500000U,
00075   k_pll_lock_poll_max  = 500000U,
00076 } clock_timeout_t;
00077
00078 /*
      ============================================================================
00079  * clock_init -- bring the chip from POR defaults to PLL 240 / PCKA 120 MHz.
00080  *
      ============================================================================
      */
00081 void clock_init(void)
00082 {
00083   volatile rx_system_regs_t *sys = system_regs();
00084
00085   /* Unlock PRC0 (clock) + PRC1 (module stop) write protection. */
00086   *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00087
00088   /* Stop sub-clock oscillator -- not used on STAR. */
00089   sys->sosccr = k_sub_clock_stopped;
00090
00091   /* Start MOSC (24 MHz external crystal). */
00092   sys->mosccr = k_main_osc_enabled;
00093
00094   /* Poll the hardware MOSC-stable flag. Exits as soon as the crystal has
00095    * settled (typically ~10 ms physical time) instead of waiting a fixed
00096    * 2.4M cycles, which on the LOCO 240 kHz boot clock is ~10 seconds. */
00097   for (volatile uint32_t i = 0; i < k_main_osc_poll_max; i++) {
00098     if ((sys->oscovfsr & k_oscovfsr_moovf) != 0U) {
00099       break;
00100     }
00101   }
00102
00103   /* Configure and enable PLL: 24 MHz x 10 / 1 = 240 MHz. */
00104   sys->pllcr  = k_pllcr_mosc_x10;
00105   sys->pllcr2 = k_pll_enabled;
00106
00107   /* Wait for PLL lock (bounded). If we time out, the crystal is missing
00108    * or not driving -- DO NOT switch SCKCR3 to PLL or the chip will halt
00109    * on a dead clock. Stay on LOCO so the LEDs still respond and the user
00110    * can see we got past clock_init. */
00111   bool pll_locked = false;
00112   for (volatile uint32_t i = 0; i < k_pll_lock_poll_max; i++) {
00113     if ((sys->oscovfsr & k_oscovfsr_plovf) != 0U) {
00114       pll_locked = true;
00115       break;
00116     }
00117   }
00118
00119   if (pll_locked) {
00120     /* MANDATORY: raise flash wait state BEFORE switching to >120 MHz. */
00121     *memwait_reg() = k_memwait_one_wait;
00122
00123     /* Apply system clock dividers. */
00124     sys->sckcr = k_sckcr_value;
00125
00126     /* Route UCLK from main PLL (UPLLSEL=0). */
00127     *((volatile uint16_t *)k_packcr_addr) = 0x0000U;
00128
00129     /* Switch system clock source to PLL. */
00130     sys->sckcr3 = k_sckcr3_cksel_pll;
00131   }
00132
00133   /* Re-lock PRCR. */
00134   *prcr_reg() = k_rx_prcr_lock;
00135 }
```

```
#include <stdint.h>
```

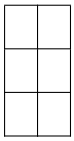


*
*
*
*
*
*
*
*
*

>

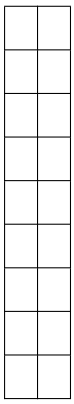
_cmt0_isr

enum cmt0_cfg_t : uint16_t



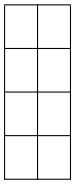
```
00043             : uint16_t {
00044     /*
00045      * CMCR layout:
00046      *   bits[1:0] CKS : 00=PCLK/8, 01=PCLK/32, 10=PCLK/128, 11=PCLK/512
00047      *   bit [6]   CMIE: 0=disabled, 1=enabled
00048      */
00049     k_cmt0_cmcr_val  = 0x0041U, /**< CKS=01 (PCLKB/32), CMIE=1 */
00050     k_cmt0_cmcor_val = 18749U,  /**< 60 MHz / 32 / 100 Hz - 1 */
00051     k_cmstr0_ch0_bit = 0x0001U,
00052 } cmt0_cfg_t;
```

enum cmt0_hw_addr_t : uintptr_t



```
00029             : uintptr_t {
00030     k_cmstr0_addr = 0x00088000U, /**< CMT start register (CH0/CH1 share) */
00031     k_cmt0_cmcr   = 0x00088002U, /**< CMT0 control register */
00032     k_cmt0_cmcnt  = 0x00088004U, /**< CMT0 counter */
00033     k_cmt0_cmcor  = 0x00088006U, /**< CMT0 compare match register */
00034
00035     k_icu_ir_base  = 0x00087000U,
00036     k_icu_ier_base = 0x00087200U,
00037     k_icu_ipr_base = 0x00087300U,
00038
00039     k_mstpcra_addr = 0x00080010U, /**< MSTPCRA (bit 15 = CMT0/CMT1 unit) */
00040     k_prcr_addr    = 0x000803FEU, /**< PRCR protection register */
00041 } cmt0_hw_addr_t;
```

enum cmt0_icu_cfg_t : uint8_t



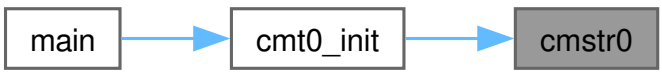
```
00054             : uint8_t {
00055     k_vect_cmt0         = 28U,
00056     k_cmt0_priority     = 3U,
00057     k_icu_ier_cmt0      = 3U, /**< IER[28 / 8] = IER[3] */
00058     k_icu_ier_cmt0_bit  = 4U, /**< 28 % 8 = 4 */
00059     k_mstpcra_cmt0_bit  = 15U,
00060 } cmt0_icu_cfg_t;
```

enum cmt0_prcr_t : uint16_t

```
00062             : uint16_t {
00063     k_prcr_unlock_prc1 = 0xA502U, /**< PRKEY=A5, PRC1=1 (CGC access) */
00064     k_prcr_lock        = 0xA500U,
00065 } cmt0_prcr_t;
```

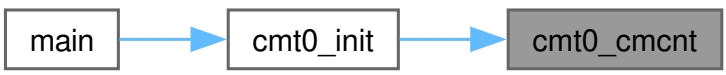
volatile uint16_t * cmstr0 (
void) [inline], [static]

```
00070 { return (volatile uint16_t *)k_cmstr0_addr; }
```



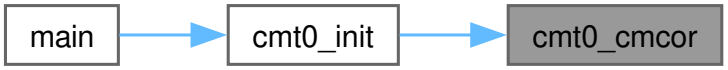
volatile uint16_t * cmt0_cmcnt (
void) [inline], [static]

```
00072 { return (volatile uint16_t *)k_cmt0_cmcnt; }
```



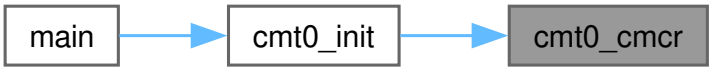
volatile uint16_t * cmt0_cmcor (
void) [inline], [static]

```
00073 { return (volatile uint16_t *)k_cmt0_cmcor; }
```



volatile uint16_t * cmt0_cmcr (
void) [inline], [static]

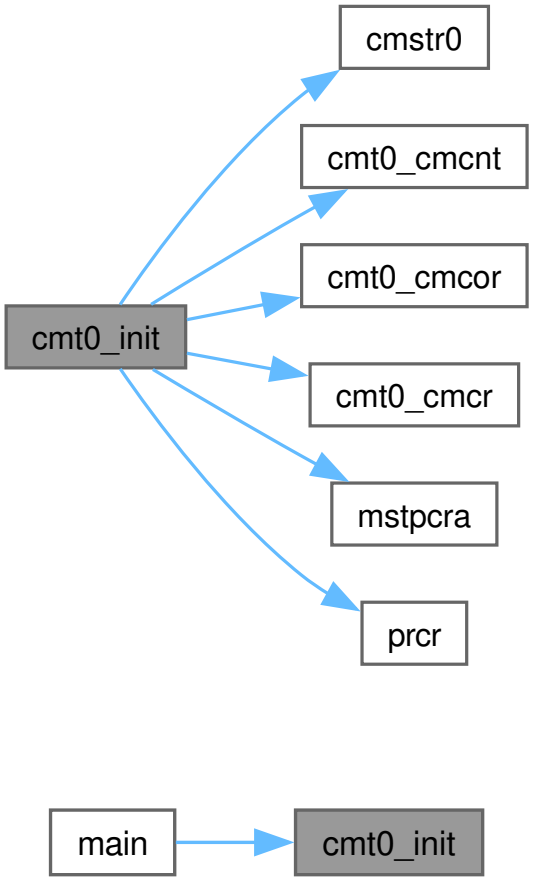
```
00071 { return (volatile uint16_t *)k_cmt0_cmcr; }
```



void cmt0_init (
void)

```
00113 {
00114     /* Release CMT0 from module stop (MSTPCRA bit 15 controls the CMT0/CMT1 unit). */
00115     *prcr() = k_prcr_unlock_prc1;
00116     *mstpcra() &= ~(uint32_t)(1UL « (uint32_t)k_mstpcra_cmt0_bit);
00117     *prcr() = k_prcr_lock;
00118
00119     /* Stop CMT0, configure, then start. */
00120     *cmstr0()    &= (uint16_t)~(uint16_t)k_cmstr0_ch0_bit;
00121     *cmt0_cmcr()  = (uint16_t)k_cmt0_cmcr_val;
00122     *cmt0_cmcor() = (uint16_t)k_cmt0_cmcor_val;
00123     *cmt0_cmcnt() = 0U;
00124
00125     /* Direct-address writes -- bypass any macro/accessor doubt.  Addresses
00126      * verified against libs/rx_hal/inc/rx72n_icu_regs.h: IR/IER/IPR are
00127      * uint8_t arrays at offsets 0x000/0x200/0x300 from ICU base 0x00087000.
00128      *   IR[28]  = 0x0008 C
00129      *   IER[3]  = 0x0008 , bit 4 = vector 28
00130      *   IPR[28] = 0x0008 C
00131      * Priority raised from 3 to 10 to match production (rx72n_icu_regs.h
00132      * example doc). */
00133     /* ICU setup for CMT0_CMI0 (vector 28).  Direct-address writes document
00134      * each register explicitly; indices verified against
00135      * libs/rx_hal/inc/rx72n_icu_regs.h (@0x000 IR, @0x200 IER, @0x300 IPR).
00136      *
00137      * IMPORTANT -- RX72N IPR INDEX IS *NOT* THE VECTOR NUMBER.
00138      * The HAL's example code `icu_regs->ipr[28] = 10‘ at
00139      * rx72n_icu_regs.h:318 is WRONG for CMT0_CMI0.  IPR[] on RX72N is
00140      * sparse-mapped per a hardware lookup table in the RX72N HW manual
00141      * section 14 (ICU).  Vector 28 (CMT0_CMI0) maps to **IPR[4]**, not
00142      * IPR[28].  Writes to IPR[28] (0x8731C) are silently dropped -- reads
```

```
00143      * return 0 regardless of what was written.  Confirmed empirically:
00144      * writing 10 to IPR[4] made the ISR fire (g_cmt0_isr_count advanced
00145      * from 0 to 604+ in ~6 sec); writing the same to IPR[28] had zero
00146      * effect.  See also eclipse-threadx/rtos-docs and Renesas community
00147      * threads referenced in the matching RCA session notes.
00148      *
00149      * We intentionally keep BOTH writes for now: IPR[4] is the real
00150      * priority register; IPR[28] is a no-op we leave in so future readers
00151      * can trivially see the wrong-address trap that cost us a session,
00152      * with this comment serving as the "why".  Remove the IPR[28] write
00153      * after this test is known-stable on hardware. */
00154     *(volatile uint8_t *)0x0008701CU = 0U;   /* IR[28]  -- clear pending */
00155     *(volatile uint8_t *)0x00087304U = 10U;  /* IPR[4]  -- real priority register for vector 28 on RX72N */
00156     *(volatile uint8_t *)0x0008731CU = 10U;  /* IPR[28] -- NO-OP, intentionally kept as a tombstone (see comment above)
        */
00157     *(volatile uint8_t *)0x00087203U |= (uint8_t)(1U << 4); /* IER[3] bit 4 enable */
00158
00159     *cmstr0() |= (uint16_t)k_cmstr0_ch0_bit;
00160
00161     /* Globally enable interrupts -- ThreadX expects PSW.I = 1 before kernel entry. */
00162     __asm__ volatile("setpsw i");
00163 }
```



volatile uint8_t * icu_ier (
uint8_t idx) [inline], [static]

```
00080 {
00081     return (volatile uint8_t *)(k_icu_ier_base + idx);
00082 }
```

volatile uint8_t * icu_ipr (
uint8_t vect) [inline], [static]

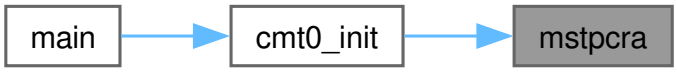
```
00084 {
00085     return (volatile uint8_t *)(k_icu_ipr_base + vect);
00086 }
```

volatile uint8_t * icu_ir (
uint8_t vect) [inline], [static]

```
00076 {
00077     return (volatile uint8_t *)(k_icu_ir_base + vect);
00078 }
```

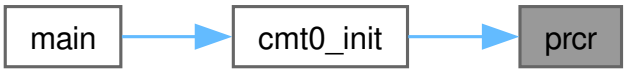
volatile uint32_t * mstpcra (
void) [inline], [static]

```
00089 {
00090     return (volatile uint32_t *)k_mstpcra_addr;
00091 }
```



volatile uint16_t * prcr (
void) [inline], [static]

```
00093 {
00094     return (volatile uint16_t *)k_prcr_addr;
00095 }
```



volatile uint32_t g_cmt0_isr_count = 0U

```
00001 /**
00002  * @file cmt0.c
00003  * @brief CMT0 ThreadX tick source for the PLL-clocked motors_rtos test.
00004  *
00005  * @details
00006  * Drives ThreadX's 100 Hz system tick via CMT0 CMI0 (vector 28).  This
00007  * variant targets the PLL clock tree (PCLKB = 60 MHz), not the LOCO
00008  * default used by blinky_rtos -- motors_rtos calls clock_init() before
00009  * tx_kernel_enter() so CMT0 sees the full 60 MHz PCLKB.
00010  *
00011  * Math: PCLKB / 32 = 60 MHz / 32 = 1.875 MHz; CMCOR = 18749 -> 100 Hz.
00012  *
00013  * The ISR must:
00014  *   1. Clear the ICU IR flag (otherwise the IRQ re-fires immediately).
00015  *   2. Call _tx_timer_interrupt() so ThreadX advances its clock.
00016  *
00017  * vectors.S wires slot 28 to `_cmt0_isr' -- GCC prefixes this file's
00018  * cmt0_isr symbol with an underscore on link, matching that reference.
00019  *
00020  * SPDX-License-Identifier: MIT
00021  * @copyright Copyright (c) 2026 Locked Inc.
00022  */
00023
00024 #include <stdint.h>
00025
00026 /*
      ============================================================================
00027  * Register addresses (RX72N HW manual R01UH0824EJ0111, Ch 15 CMT + Ch 13 ICU)
00028  *
      ============================================================================
      */
00029 typedef enum : uintptr_t {
00030     k_cmstr0_addr = 0x00088000U, /**< CMT start register (CH0/CH1 share) */
00031     k_cmt0_cmcr   = 0x00088002U, /**< CMT0 control register */
00032     k_cmt0_cmcnt  = 0x00088004U, /**< CMT0 counter */
00033     k_cmt0_cmcor  = 0x00088006U, /**< CMT0 compare match register */
00034
00035     k_icu_ir_base  = 0x00087000U,
00036     k_icu_ier_base = 0x00087200U,
00037     k_icu_ipr_base = 0x00087300U,
00038
00039     k_mstpcra_addr = 0x00080010U, /**< MSTPCRA (bit 15 = CMT0/CMT1 unit) */
00040     k_prcr_addr    = 0x000803FEU, /**< PRCR protection register */
00041 } cmt0_hw_addr_t;
00042
00043 typedef enum : uint16_t {
00044     /*
00045      * CMCR layout:
00046      *   bits[1:0] CKS : 00=PCLK/8, 01=PCLK/32, 10=PCLK/128, 11=PCLK/512
00047      *   bit [6]   CMIE: 0=disabled, 1=enabled
00048      */
00049     k_cmt0_cmcr_val  = 0x0041U, /**< CKS=01 (PCLKB/32), CMIE=1 */
00050     k_cmt0_cmcor_val = 18749U,  /**< 60 MHz / 32 / 100 Hz - 1 */
00051     k_cmstr0_ch0_bit = 0x0001U,
00052 } cmt0_cfg_t;
00053
00054 typedef enum : uint8_t {
00055     k_vect_cmt0         = 28U,
00056     k_cmt0_priority     = 3U,
00057     k_icu_ier_cmt0      = 3U, /**< IER[28 / 8] = IER[3] */
00058     k_icu_ier_cmt0_bit  = 4U, /**< 28 % 8 = 4 */
00059     k_mstpcra_cmt0_bit  = 15U,
00060 } cmt0_icu_cfg_t;
00061
00062 typedef enum : uint16_t {
00063     k_prcr_unlock_prc1 = 0xA502U, /**< PRKEY=A5, PRC1=1 (CGC access) */
00064     k_prcr_lock        = 0xA500U,
00065 } cmt0_prcr_t;
00066
00067 /*
      ============================================================================
00068  * Register accessors
00069  *
      ============================================================================
      */
```

```
00070 static inline volatile uint16_t *cmstr0(void)    { return (volatile uint16_t *)k_cmstr0_addr; }
00071 static inline volatile uint16_t *cmt0_cmcr(void) { return (volatile uint16_t *)k_cmt0_cmcr; }
00072 static inline volatile uint16_t *cmt0_cmcnt(void){ return (volatile uint16_t *)k_cmt0_cmcnt; }
00073 static inline volatile uint16_t *cmt0_cmcor(void){ return (volatile uint16_t *)k_cmt0_cmcor; }
00074
00075 static inline volatile uint8_t *icu_ir(uint8_t vect)
00076 {
00077     return (volatile uint8_t *)(k_icu_ir_base + vect);
00078 }
00079 static inline volatile uint8_t *icu_ier(uint8_t idx)
00080 {
00081     return (volatile uint8_t *)(k_icu_ier_base + idx);
00082 }
00083 static inline volatile uint8_t *icu_ipr(uint8_t vect)
00084 {
00085     return (volatile uint8_t *)(k_icu_ipr_base + vect);
00086 }
00087
00088 static inline volatile uint32_t *mstpcra(void)
00089 {
00090     return (volatile uint32_t *)k_mstpcra_addr;
00091 }
00092 static inline volatile uint16_t *prcr(void)
00093 {
00094     return (volatile uint16_t *)k_prcr_addr;
00095 }
00096
00097
00098 /*
    ============================================================================
00099  * CMT0 ISR (wired at vectors.S slot 28).  GCC's __attribute__((interrupt))
00100  * generates the RTE-terminated prologue/epilogue the RX expects.
00101  *
    ============================================================================
    */
00102 /* Exposed to main.c for the diagnostic CSV column -- counts how many times
00103  * the CMT0 compare-match ISR has fired.  Incremented by the assembly ISR
00104  * in vectors.S (which wraps _tx_timer_interrupt in context_save/restore so
00105  * that timer-based task wakeups actually cause a context switch).  See the
00106  * `_cmt0_isr:` block in vectors.S for the full ISR implementation. */
00107 volatile uint32_t g_cmt0_isr_count = 0U;
00108
00109 /*
    ============================================================================
00110  * cmt0_init -- call once, before tx_kernel_enter().
00111  *
    ============================================================================
    */
00112 void cmt0_init(void)
00113 {
00114     /* Release CMT0 from module stop (MSTPCRA bit 15 controls the CMT0/CMT1 unit). */
00115     *prcr() = k_prcr_unlock_prc1;
00116     *mstpcra() &= ~(uint32_t)(1UL << (uint32_t)k_mstpcra_cmt0_bit);
00117     *prcr() = k_prcr_lock;
00118
00119     /* Stop CMT0, configure, then start. */
00120     *cmstr0()    &= (uint16_t)~(uint16_t)k_cmstr0_ch0_bit;
00121     *cmt0_cmcr()  = (uint16_t)k_cmt0_cmcr_val;
00122     *cmt0_cmcor() = (uint16_t)k_cmt0_cmcor_val;
00123     *cmt0_cmcnt() = 0U;
00124
00125     /* Direct-address writes -- bypass any macro/accessor doubt.  Addresses
00126      * verified against libs/rx_hal/inc/rx72n_icu_regs.h: IR/IER/IPR are
00127      * uint8_t arrays at offsets 0x000/0x200/0x300 from ICU base 0x00087000.
00128      *   IR[28]  = 0x0008 C
00129      *   IER[3]  = 0x0008 , bit 4 = vector 28
00130      *   IPR[28] = 0x0008 C
00131      * Priority raised from 3 to 10 to match production (rx72n_icu_regs.h
00132      * example doc). */
00133     /* ICU setup for CMT0_CMI0 (vector 28).  Direct-address writes document
00134      * each register explicitly; indices verified against
00135      * libs/rx_hal/inc/rx72n_icu_regs.h (@0x000 IR, @0x200 IER, @0x300 IPR).
00136      *
00137      * IMPORTANT -- RX72N IPR INDEX IS *NOT* THE VECTOR NUMBER.
00138      * The HAL's example code `icu_regs->ipr[28] = 10` at
00139      * rx72n_icu_regs.h:318 is WRONG for CMT0_CMI0.  IPR[] on RX72N is
00140      * sparse-mapped per a hardware lookup table in the RX72N HW manual
00141      * section 14 (ICU).  Vector 28 (CMT0_CMI0) maps to **IPR[4]**, not
00142      * IPR[28].  Writes to IPR[28] (0x8731C) are silently dropped -- reads
00143      * return 0 regardless of what was written.  Confirmed empirically:
00144      * writing 10 to IPR[4] made the ISR fire (g_cmt0_isr_count advanced
00145      * from 0 to 604+ in ~6 sec); writing the same to IPR[28] had zero
00146      * effect.  See also eclipse-threadx/rtos-docs and Renesas community
00147      * threads referenced in the matching RCA session notes.
00148      *
00149      * We intentionally keep BOTH writes for now: IPR[4] is the real
00150      * priority register; IPR[28] is a no-op we leave in so future readers
```

```
00151      * can trivially see the wrong-address trap that cost us a session,
00152      * with this comment serving as the "why".  Remove the IPR[28] write
00153      * after this test is known-stable on hardware. */
00154     *(volatile uint8_t *)0x0008701CU = 0U;   /* IR[28]  -- clear pending */
00155     *(volatile uint8_t *)0x00087304U = 10U;  /* IPR[4]  -- real priority register for vector 28 on RX72N */
00156     *(volatile uint8_t *)0x0008731CU = 10U;  /* IPR[28] -- NO-OP, intentionally kept as a tombstone (see comment above)
    */
00157     *(volatile uint8_t *)0x00087203U |= (uint8_t)(1U << 4); /* IER[3] bit 4 enable */
00158
00159     *cmstr0() |= (uint16_t)k_cmstr0_ch0_bit;
00160
00161     /* Globally enable interrupts -- ThreadX expects PSW.I = 1 before kernel entry. */
00162     __asm__ volatile("setpsw i");
00163 }
```

```
00001 /*
00002  * blinky_rtos/linker.ld -- Linker script for ThreadX-based multi-LED breathe.
00003  *
00004  * RX72N memory map (R5F572NNHxFB):
00005  *   Internal RAM  : 0x00000004 - 0x0007FFFF  (512 KB, first 4 bytes reserved)
00006  *   Internal ROM  : 0xFFE00000 - 0xFFFFFFFF  (2 MB)
00007  *
00008  * Fixed hardware vector locations:
00009  *   0xFFFFFF80 - 0xFFFFFFFC  Exception vector table (EXTB)
00010  *   0xFFFFFFFC - 0xFFFFFFFF  Reset vector (points to startup entry)
00011  *
00012  * INTB is loaded by startup.S to point at _rvectors_start (in ROM).
00013  *
00014  * Renesas-syntax sections from ThreadX RXv3 port (tx_timer_interrupt.S etc.):
00015  *   P, P   -- code sections (.SECTION P,CODE)
00016  *   B, B   -- BSS sections  (.SECTION B,DATA)
00017  *   D, D   -- initialised data sections
00018  */
00019
00020 MEMORY
00021 {
00022     RAM : ORIGIN = 0x00000004, LENGTH = 0x7FFFC   /* 512 KB - 4 bytes */
00023     ROM : ORIGIN = 0xFFE00000, LENGTH = 0x200000  /* 2 MB             */
00024 }
00025
00026 SECTIONS
00027 {
00028     /* Code -- explicit ROM base so section VMA is anchored correctly.
00029      * *(P) captures Renesas-assembler code sections from ThreadX port files. */
00030     .text 0xFFE00000 : AT(0xFFE00000)
00031     {
00032         *(.text*)
00033         *(P)
00034         *(P )
00035         *(.rodata*)
00036         *(C)
00037         *(C )
00038         *(C )
00039         etext = .;
00040     } > ROM
00041
00042     /*
00043      * Relocatable vector table -- startup.S loads INTB with this address.
00044      * Must be 4-byte aligned; 32 entries = 128 bytes.
00045      * No AT() needed: follows .text in ROM, VMA and LMA are the same.
00046      */
00047     .rvectors ALIGN(4) :
00048     {
00049         _rvectors_start = .;
00050         KEEP(*(.rvectors))
00051         _rvectors_end = .;
00052     } > ROM
00053
00054     /*
00055      * Initialised data -- LMA stays in ROM (image stored here), VMA in RAM.
00056      * startup.S copies from _mdata (ROM) to _data..._edata (RAM) on boot.
00057      * Required for ThreadX: ___tx_thread_system_state starts as 0xF0F0F0F0.
00058      */
00059     _mdata = .;
00060     .data : AT(_mdata)
00061     {
00062         _data = .;
00063         *(.data*)
00064         *(D)
00065         *(D )
00066         *(D )
00067         _edata = .;
```

```
00068     } > RAM
00069
00070     /* BSS -- ThreadX needs this zeroed; startup.S handles it.
00071      * B/B /B  are Renesas-assembler BSS sections from ThreadX port files. */
00072     .bss :
00073     {
00074         _bss = .;
00075         *(.bss*)
00076         *(COMMON)
00077         *(B)
00078         *(B )
00079         *(B )
00080         _ebss = .;
00081         . = ALIGN(4);
00082         _end = .;          /* ThreadX tx_initialize_low_level reads _end */
00083     } > RAM AT>RAM
00084
00085     /*
00086      * Stacks -- USP grows down from top of RAM, ISP 2 KB below that.
00087      * ThreadX uses the interrupt stack pointer (ISP) for ISR execution.
00088      */
00089     _ustack = ORIGIN(RAM) + LENGTH(RAM);
00090     _istack = _ustack - 0x800;
00091
00092     /*
00093      * Exception vector table (EXTB) at fixed hardware address 0xFFFFFF80.
00094      */
00095     .exvectors 0xFFFFFF80 : AT(0xFFFFFF80)
00096     {
00097         LONG(0xFFFFFFFF);   /* BUSERR */
00098         LONG(0xFFFFFFFF);   /* reserved */
00099         LONG(0xFFFFFFFF);   /* FIFERR */
00100         LONG(0xFFFFFFFF);   /* FRDYI */
00101         LONG(0xFFFFFFFF);   /* reserved */
00102         LONG(0xFFFFFFFF);
00103         LONG(0xFFFFFFFF);
00104         LONG(0xFFFFFFFF);
00105         LONG(0xFFFFFFFF);
00106         LONG(0xFFFFFFFF);
00107         LONG(0xFFFFFFFF);
00108         LONG(0xFFFFFFFF);
00109         LONG(0xFFFFFFFF);
00110         LONG(0xFFFFFFFF);
00111         LONG(0xFFFFFFFF);
00112         LONG(0xFFFFFFFF);
00113         LONG(0xFFFFFFFF);
00114         LONG(0xFFFFFFFF);
00115         LONG(0xFFFFFFFF);
00116         LONG(0xFFFFFFFF);
00117         LONG(0xFFFFFFFF);
00118         LONG(0xFFFFFFFF);
00119         LONG(0xFFFFFFFF);
00120         LONG(0xFFFFFFFF);
00121         LONG(0xFFFFFFFF);
00122         LONG(0xFFFFFFFF);
00123         LONG(0xFFFFFFFF);
00124         LONG(0xFFFFFFFF);
00125         LONG(0xFFFFFFFF);
00126         LONG(0xFFFFFFFF);
00127         LONG(0xFFFFFFFF);
00128     } > ROM
00129
00130     /* Fixed reset vector at 0xFFFFFFFC */
00131     .fvectors 0xFFFFFFFC : AT(0xFFFFFFFC)
00132     {
00133         KEEP(*(.fvectors))
00134     } > ROM
00135 }
```

```
#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>
#include "tx_api.h"
#include "rx_motor.h"
#include "rx_gptw.h"
#include "rx_mpc.h"
#include "rx_port_constants.h"
#include "rx_mtu_encoder.h"
#include "rx_encoder_tpu.h"
```

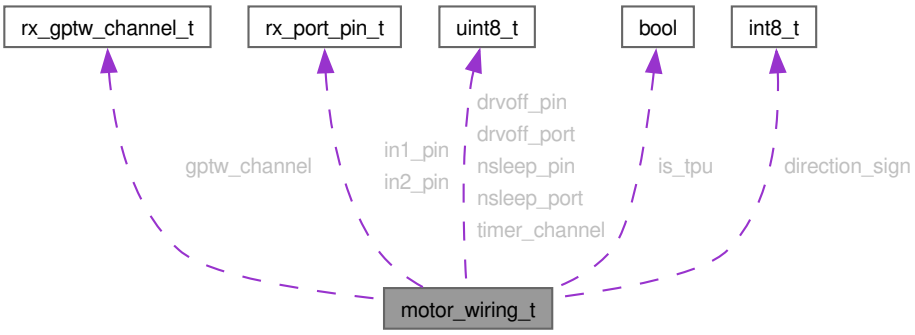
#include ”rx72n_mtu_regs.h”
#include ”rx72n_tpu_regs.h”
#include ”rx72n_system_regs.h”
#include ”rx72n_icu_regs.h”
#include ”rx_register_protection.h”

*

*
*
*

*
*

*



#define REG16__AT(
a)

(*(volatile uint16_t *)(uintptr_t)(a))

#define REG8(
a)

(*(volatile uint8_t *)(uintptr_t)(a))

```
#define REG8_AT(
                        a)


 (*(volatile uint8_t  *)(uintptr_t)(a))
```

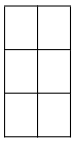
```
enum delay_const_t : uint32_t
```

```
 00200             : uint32_t {
 00201     k_iters_per_ms = 40000U,
 00202 } delay_const_t;
```

```
enum motor_const_t : uint8_t
```

```
 00172             : uint8_t {
 00173     k_motor_count = 4,
 00174 } motor_const_t;
```

```
enum port_reg_base_t : uintptr_t
```

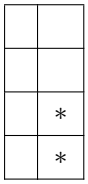


```
 00073             : uintptr_t {
 00074     k_port_pdr_base  = 0x0008C000U,
 00075     k_port_podr_base = 0x0008C020U,
 00076     k_port_pmr_base  = 0x0008C060U,
 00077 } port_reg_base_t;
```

```
enum pwm_const_t : uint32_t
```

```
 00235             : uint32_t {
 00236     k_pwm_freq_hz  = 20000U,
 00237     k_deadtime_ns  = 1000U,
 00238 } pwm_const_t;
```

enum rtos_prio_t : uint8_t

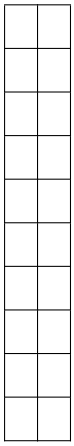


```
00406             : uint8_t {
00407     k_encoder_task_priority = 9U,  /**< higher priority: prints CSV, keeps console responsive */
00408     k_motor_task_priority   = 10U, /**< lower priority: sweeps duty, preempted by encoder */
00409     k_motor_sleep_ticks     = 50U, /**< 50 * 10 ms = 500 ms */
00410     k_encoder_sleep_ticks   = 10U, /**< 10 * 10 ms = 100 ms */
00411 } rtos_prio_t;
```

enum rtos_stack_t : uint16_t

```
00401             : uint16_t {
00402     k_motor_task_stack_sz   = 2048U,
00403     k_encoder_task_stack_sz = 2048U,
00404 } rtos_stack_t;
```

enum status_led_pins_t : uint8_t



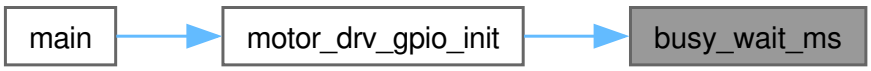
```
00113             : uint8_t {
00114     k_port    = 7,
00115     k_port_a  = 10,
00116     k_port_b  = 11,
00117     k_port_e  = 14,
00118     k_port    = 6,
00119
00120     k_bit_led_heartbeat = 7, /**< PA7 -- D9, toggled 10 Hz by encoder_task */
00121     k_bit_led_init_done = 0, /**< PB0 -- D10, solid once main() finishes init */
00122     k_bit_led_motor_run = 1, /**< P71 -- D11, toggles on every motor_task step */
00123     k_bit_led_enc_tick  = 2, /**< P72 -- D12, toggles per encoder sample */
00124     k_bit_led_motor_alive = 1, /**< PB1 -- D13, proves motor_task reached loop body */
00125 } status_led_pins_t;
```

enum sweep_const_t : uint8_t

```
00388             : uint8_t {
00389     k_sweep_step_count = 21U, /**< -100, -90, ..., 0, ..., +100 */
00390 } sweep_const_t;
```

void busy_wait_ms (
uint32_t ms) [static]

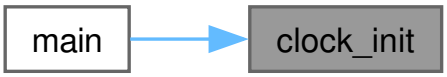
```
00205 {
00206     for (uint32_t m = 0; m < ms; m++) {
00207         for (volatile uint32_t i = 0; i < k_iters_per_ms; i++) {
00208             ___asm___ volatile("nop");
00209         }
00210     }
00211 }
```



void clock_init (
void) [extern]

```
00082 {
00083   volatile rx_system_regs_t *sys = system_regs();
00084
00085   /* Unlock PRC0 (clock) + PRC1 (module stop) write protection. */
00086   *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00087
00088   /* Stop sub-clock oscillator -- not used on STAR. */
00089   sys->sosccr = k_sub_clock_stopped;
00090
00091   /* Start MOSC (24 MHz external crystal). */
00092   sys->mosccr = k_main_osc_enabled;
00093
00094   /* Poll the hardware MOSC-stable flag. Exits as soon as the crystal has
00095    * settled (typically ~10 ms physical time) instead of waiting a fixed
00096    * 2.4M cycles, which on the LOCO 240 kHz boot clock is ~10 seconds. */
00097   for (volatile uint32_t i = 0; i < k_main_osc_poll_max; i++) {
00098     if ((sys->oscovfsr & k_oscovfsr_moovf) != 0U) {
00099       break;
00100     }
00101   }
00102
00103   /* Configure and enable PLL: 24 MHz x 10 / 1 = 240 MHz. */
00104   sys->pllcr  = k_pllcr_mosc_x10;
00105   sys->pllcr2 = k_pll_enabled;
00106
00107   /* Wait for PLL lock (bounded). If we time out, the crystal is missing
00108    * or not driving -- DO NOT switch SCKCR3 to PLL or the chip will halt
00109    * on a dead clock. Stay on LOCO so the LEDs still respond and the user
00110    * can see we got past clock_init. */
00111   bool pll_locked = false;
00112   for (volatile uint32_t i = 0; i < k_pll_lock_poll_max; i++) {
```

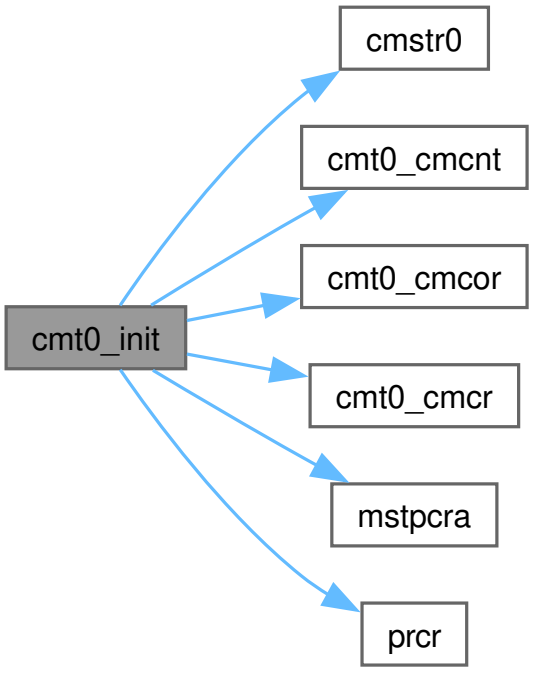
```
00113     if ((sys->oscovfsr & k_oscovfsr_plovf) != 0U) {
00114       pll_locked = true;
00115       break;
00116     }
00117   }
00118
00119   if (pll_locked) {
00120     /* MANDATORY: raise flash wait state BEFORE switching to >120 MHz. */
00121     *memwait_reg() = k_memwait_one_wait;
00122
00123     /* Apply system clock dividers. */
00124     sys->sckcr = k_sckcr_value;
00125
00126     /* Route UCLK from main PLL (UPLLSEL=0). */
00127     *((volatile uint16_t *)k_packcr_addr) = 0x0000U;
00128
00129     /* Switch system clock source to PLL. */
00130     sys->sckcr3 = k_sckcr3_cksel_pll;
00131   }
00132
00133   /* Re-lock PRCR. */
00134   *prcr_reg() = k_rx_prcr_lock;
00135 }
```



void cmt0_init (
void) [extern]

```
00113 {
00114     /* Release CMT0 from module stop (MSTPCRA bit 15 controls the CMT0/CMT1 unit). */
00115     *prcr() = k_prcr_unlock_prc1;
00116     *mstpcra() &= ~(uint32_t)(1UL « (uint32_t)k_mstpcra_cmt0_bit);
00117     *prcr() = k_prcr_lock;
00118
00119     /* Stop CMT0, configure, then start. */
00120     *cmstr0()    &= (uint16_t)~(uint16_t)k_cmstr0_ch0_bit;
00121     *cmt0_cmcr()  = (uint16_t)k_cmt0_cmcr_val;
00122     *cmt0_cmcor() = (uint16_t)k_cmt0_cmcor_val;
00123     *cmt0_cmcnt() = 0U;
00124
00125     /* Direct-address writes -- bypass any macro/accessor doubt.  Addresses
00126      * verified against libs/rx_hal/inc/rx72n_icu_regs.h: IR/IER/IPR are
00127      * uint8_t arrays at offsets 0x000/0x200/0x300 from ICU base 0x00087000.
00128      *   IR[28]  = 0x0008 C
00129      *   IER[3]  = 0x0008 , bit 4 = vector 28
00130      *   IPR[28] = 0x0008 C
00131      * Priority raised from 3 to 10 to match production (rx72n_icu_regs.h
00132      * example doc). */
00133     /* ICU setup for CMT0_CMI0 (vector 28).  Direct-address writes document
00134      * each register explicitly; indices verified against
00135      * libs/rx_hal/inc/rx72n_icu_regs.h (@0x000 IR, @0x200 IER, @0x300 IPR).
00136      *
00137      * IMPORTANT -- RX72N IPR INDEX IS *NOT* THE VECTOR NUMBER.
00138      * The HAL's example code `icu_regs->ipr[28] = 10‘ at
00139      * rx72n_icu_regs.h:318 is WRONG for CMT0_CMI0.  IPR[] on RX72N is
00140      * sparse-mapped per a hardware lookup table in the RX72N HW manual
00141      * section 14 (ICU).  Vector 28 (CMT0_CMI0) maps to **IPR[4]**, not
00142      * IPR[28].  Writes to IPR[28] (0x8731C) are silently dropped -- reads
00143      * return 0 regardless of what was written.  Confirmed empirically:
00144      * writing 10 to IPR[4] made the ISR fire (g_cmt0_isr_count advanced
00145      * from 0 to 604+ in ~6 sec); writing the same to IPR[28] had zero
00146      * effect.  See also eclipse-threadx/rtos-docs and Renesas community
00147      * threads referenced in the matching RCA session notes.
00148      *
00149      * We intentionally keep BOTH writes for now: IPR[4] is the real
00150      * priority register; IPR[28] is a no-op we leave in so future readers
00151      * can trivially see the wrong-address trap that cost us a session,
```

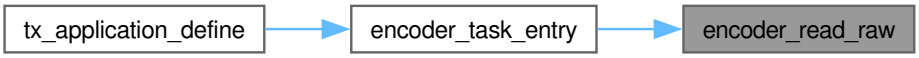
```
00152      * with this comment serving as the "why".  Remove the IPR[28] write
00153      * after this test is known-stable on hardware. */
00154     *(volatile uint8_t *)0x0008701CU = 0U;   /* IR[28]  -- clear pending */
00155     *(volatile uint8_t *)0x00087304U = 10U;  /* IPR[4]  -- real priority register for vector 28 on RX72N */
00156     *(volatile uint8_t *)0x0008731CU = 10U;  /* IPR[28] -- NO-OP, intentionally kept as a tombstone (see comment above)
       */
00157     *(volatile uint8_t *)0x00087203U |= (uint8_t)(1U << 4); /* IER[3] bit 4 enable */
00158
00159     *cmstr0() |= (uint16_t)k_cmstr0_ch0_bit;
00160
00161     /* Globally enable interrupts -- ThreadX expects PSW.I = 1 before kernel entry. */
00162     __asm__ volatile("setpsw i");
00163 }
```



uint16_t encoder_read_raw (
uint8_t motor_idx) [static]

```
00336 {
00337     /* IMPORTANT -- indices 2 and 3 are INTENTIONALLY SWAPPED relative to the
00338      * k_motors[] table's `.timer_channel` field and the previous "M2->TPU1,
00339      * M3->TPU2" comment.  Cross-check against motor_spin_test IPROPI on
00340      * 2026-04-21 showed M2 (BR) drawing ~0 current at every duty (dead
00341      * motor, hardware issue) while the encoder at TPU1 was counting fast
00342      * and the encoder at TPU2 was pinned at 0.  The only consistent reading
00343      * is that the physical encoder harness wires BR's encoder to TPU2
00344      * (TCLKC/TCLKD on PC0/PB3) and BL's encoder to TPU1 (TCLKA/TCLKB on
00345      * PC2/PA3) -- the opposite of what the k_motors[] comment claims.
```

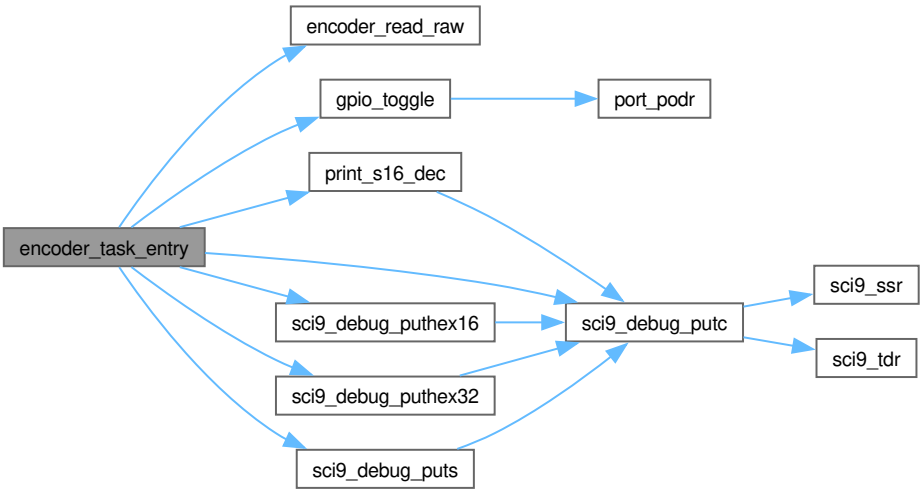
```
00346     * Swapping the indices here makes m2 track BR and m3 track BL so the
00347     * CSV column matches the motor driver index in motor_spin_test. */
00348    static const uintptr_t k_tcnt_addr[k_motor_count] = {
00349        0x000C1386U,       /* MTU1 -- M0 (FL) */
00350        0x000C1406U,       /* MTU2 -- M1 (FR) */
00351        0x00088130U + 6U,  /* TPU2 -- M2 (BR): harness wires BR -> TPU2 */
00352        0x00088120U + 6U,  /* TPU1 -- M3 (BL): harness wires BL -> TPU1 */
00353    };
00354    return REG16_AT(k_tcnt_addr[motor_idx]);
00355 }
```



void encoder_task_entry (
ULONG arg) [static]

```
00487 {
00488    (void)arg;
00489
00490    sci9_debug_puts("t_ms,cmt0_cnt,mot_state,m_ent,duty,m0,m1,m2,m3\n");
00491
00492    uint32_t t_ms = 0U;
00493
00494    for (;;) {
00495        gpio_toggle(k_port_a, k_bit_led_heartbeat);
00496        gpio_toggle(k_port , k_bit_led_enc_tick);
00497
00498        const uint16_t c0 = encoder_read_raw(0U);
00499        const uint16_t c1 = encoder_read_raw(1U);
00500        const uint16_t c2 = encoder_read_raw(2U);
00501        const uint16_t c3 = encoder_read_raw(3U);
00502
00503
00504        print_s16_dec((int16_t)(t_ms & 0x7FFFU));
00505        sci9_debug_putc(',');
00506        print_s16_dec((int16_t)(g_cmt0_isr_count & 0x7FFFU));
00507        sci9_debug_putc(',');
00508        print_s16_dec((int16_t)s_motor_tcb.tx_thread_state);
00509        sci9_debug_putc(',');
00510        print_s16_dec((int16_t)g_motor_task_entries);
00511        sci9_debug_putc(',');
00512        print_s16_dec((int16_t)s_current_duty);
00513        sci9_debug_putc(',');
00514        print_s16_dec((int16_t)c0);
00515        sci9_debug_putc(',');
00516        print_s16_dec((int16_t)c1);
00517        sci9_debug_putc(',');
00518        print_s16_dec((int16_t)c2);
00519        sci9_debug_putc(',');
00520        print_s16_dec((int16_t)c3);
00521        sci9_debug_putc('\n');
00522
00523        /* Option 1b diagnostic: capture all TX_CALLER_ERROR discriminators
00524         * BEFORE sleep plus its return value AFTER.  ss and cur distinguish
00525         * the three remaining TX_CALLER_ERROR paths in tx_thread_sleep.c. */
00526        const UINT        pd  = _tx_thread_preempt_disable;
00527        const ULONG       ss  = _tx_thread_system_state;
00528        const TX_THREAD  *cur = _tx_thread_current_ptr;
00529        const UINT        ret = tx_thread_sleep((ULONG)k_encoder_sleep_ticks);
00530        sci9_debug_puts("pd=");
00531        print_s16_dec((int16_t)pd);
00532        sci9_debug_puts(" ss=");
00533        sci9_debug_puthex32((uint32_t)ss);
00534        sci9_debug_puts(" cur=");
00535        sci9_debug_puthex32((uint32_t)(uintptr_t)cur);
00536        sci9_debug_puts(" ret=");
00537        sci9_debug_puthex16((uint16_t)ret);
00538        sci9_debug_putc('\n');
00539        t_ms += 100U;
00540    }
```

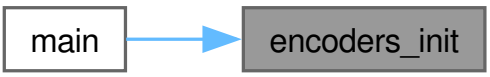
```
00541 }
```



bool encoders_init (void) [static]

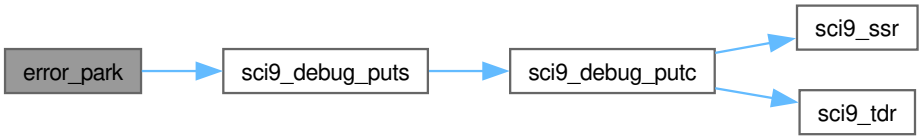
```
00271 {
00272     /* Step 1: release MTU (MSTPA9) + TPU (MSTPA13) module clocks. */
00273     *prcr_reg() = k_rx_prcr_unlock_all;
00274     system_regs()->mstpcra &= ~(uint32_t)((1UL << 9) | (uint32_t)k_tpu_mstpcra_mstpa13);
00275     *prcr_reg() = k_rx_prcr_lock;
00276
00277     /* Step 2 + 3: MPC pin sequence PMR=0, write PFS, PMR=1 for every
00278      * encoder phase input. */
00279     volatile uint8_t *pwpr  = (volatile uint8_t *)0x0008C11FU;
00280     volatile uint8_t *p2pmr = (volatile uint8_t *)0x0008C062U;
00281     volatile uint8_t *papmr = (volatile uint8_t *)0x0008C06AU;
00282     volatile uint8_t *pbpmr = (volatile uint8_t *)0x0008C06BU;
00283     volatile uint8_t *pcpmr = (volatile uint8_t *)0x0008C06CU;
00284
00285     *p2pmr &= (uint8_t)~(uint8_t)((1U << 4) | (1U << 5));
00286     *papmr &= (uint8_t)~(uint8_t)((1U << 1) | (1U << 3));
00287     *pcpmr &= (uint8_t)~(uint8_t)((1U << 0) | (1U << 2) | (1U << 5));
00288     *pbpmr &= (uint8_t)~(uint8_t)(1U << 3);
00289
00290     *pwpr = 0x00U;
00291     *pwpr = 0x40U;
00292     REG8_AT(0x0008C140U + 2U * 8U + 4U)  = 0x02U; /* P24 MTCLKA */
00293     REG8_AT(0x0008C140U + 2U * 8U + 5U)  = 0x02U; /* P25 MTCLKB */
00294     REG8_AT(0x0008C140U + 10U * 8U + 1U) = 0x02U; /* PA1 MTCLKC */
00295     REG8_AT(0x0008C140U + 12U * 8U + 5U) = 0x02U; /* PC5 MTCLKD */
00296     REG8_AT(0x0008C140U + 12U * 8U + 2U) = 0x03U; /* PC2 TCLKA  */
00297     REG8_AT(0x0008C140U + 10U * 8U + 3U) = 0x04U; /* PA3 TCLKB  */
00298     REG8_AT(0x0008C140U + 12U * 8U + 0U) = 0x03U; /* PC0 TCLKC  */
00299     REG8_AT(0x0008C140U + 11U * 8U + 3U) = 0x04U; /* PB3 TCLKD  */
00300     *pwpr = 0x00U;
00301     *pwpr = 0x80U;
00302
```

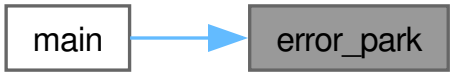
```
00303     *p2pmr |= (uint8_t)((1U « 4) | (1U « 5));
00304     *papmr |= (uint8_t)((1U « 1) | (1U « 3));
00305     *pcpmr |= (uint8_t)((1U « 0) | (1U « 2) | (1U « 5));
00306     *pbpmr |= (uint8_t)(1U « 3);
00307
00308     /* Phase counting mode 1 on MTU1 + MTU2 */
00309     volatile rx_mtu_channel_regs_t *m1 = mtu1();
00310     m1->tcr = 0;  m1->tmdr = 0x04U;
00311     m1->tiorh = 0; m1->tiorl = 0;
00312     m1->tier = 0;  m1->tsr = 0;  m1->tcnt = 0;
00313     volatile rx_mtu_channel_regs_t *m2 = mtu2();
00314     m2->tcr = 0;  m2->tmdr = 0x04U;
00315     m2->tiorh = 0; m2->tiorl = 0;
00316     m2->tier = 0;  m2->tsr = 0;  m2->tcnt = 0;
00317
00318     /* Phase counting mode 1 on TPU1 + TPU2 */
00319     volatile rx_tpu_regs_t *t1 = tpu1();
00320     t1->tcr = 0;  t1->tmdr = 0x04U;  t1->tcnt = 0;
00321     volatile rx_tpu_regs_t *t2 = tpu2();
00322     t2->tcr = 0;  t2->tmdr = 0x04U;  t2->tcnt = 0;
00323
00324     /* Clear TPU noise filters from any prior-run leftover state */
00325     tpu_control()->nfcr[1] = 0;
00326     tpu_control()->nfcr[2] = 0;
00327
00328     /* Start all four counters */
00329     mtu_tstra()->tstr   |= (uint8_t)(k_mtu_tstr_cst1 | k_mtu_tstr_cst2);
00330     tpu_control()->tstr |= (uint8_t)(k_tpu_tstr_cst1 | k_tpu_tstr_cst2);
00331
00332     return true;
00333 }
```



void error_park (
const char * msg) [static]

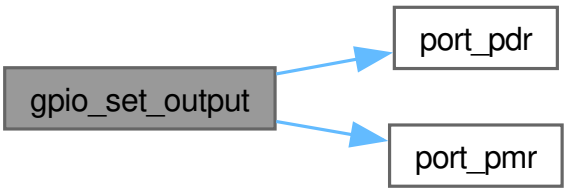
```
00569 {
00570     sci9_debug_puts("FATAL ");
00571     sci9_debug_puts(msg);
00572     sci9_debug_puts("\n");
00573
00574     /* Best-effort motor safe: attempt a 0-duty write (may fail if rx_motor
00575      * isn't initialised, but that's OK since GPTW would be idle in that case). */
00576     for (uint8_t i = 0; i < k_motor_count; i++) {
00577         (void)rx_motor_set_duty(&s_motors[i], 0.0F);
00578     }
00579
00580     for (;;) {
00581         __asm__ volatile("nop");
00582     }
00583 }
```

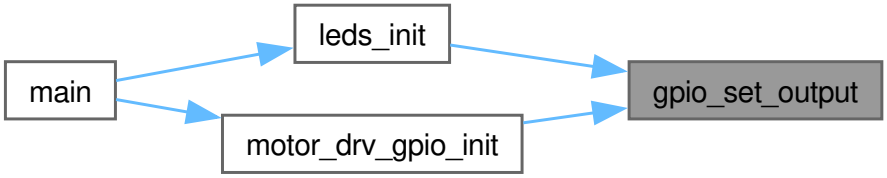




void gpio_set_output (
uint8_t port,
uint8_t pin) [inline], [static]

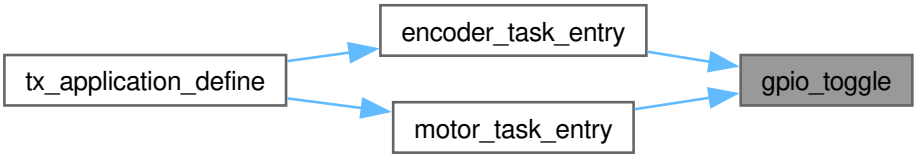
```
00093 {
00094     *port_pmr(port) &= (uint8_t) ~(1U « pin);
00095     *port_pdr(port) |= (uint8_t)(1U « pin);
00096 }
```





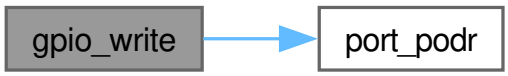
void gpio_toggle (
uint8_t port,
uint8_t pin) [inline], [static]

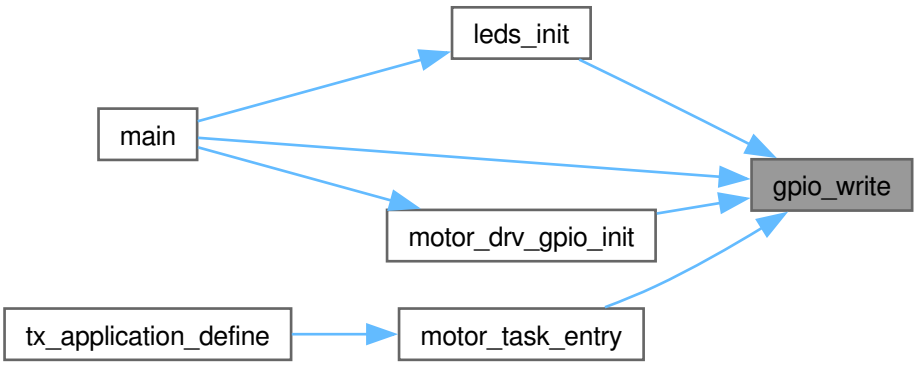
```
00106 {
00107     *port_podr(port) ^= (uint8_t)(1U « pin);
00108 }
```



void gpio_write (
uint8_t port,
uint8_t pin,
bool high) [inline], [static]

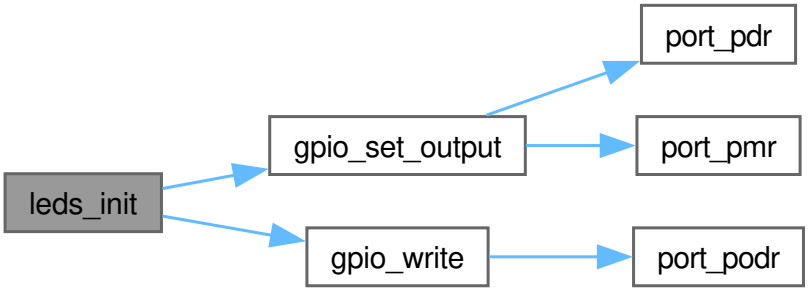
```
00098 {
00099     if (high) {
00100         *port_podr(port) |= (uint8_t)(1U « pin);
00101     } else {
00102         *port_podr(port) &= (uint8_t) ~(1U « pin);
00103     }
00104 }
```

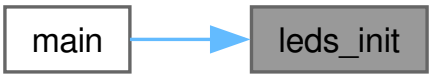




void leds_init (
void) [static]

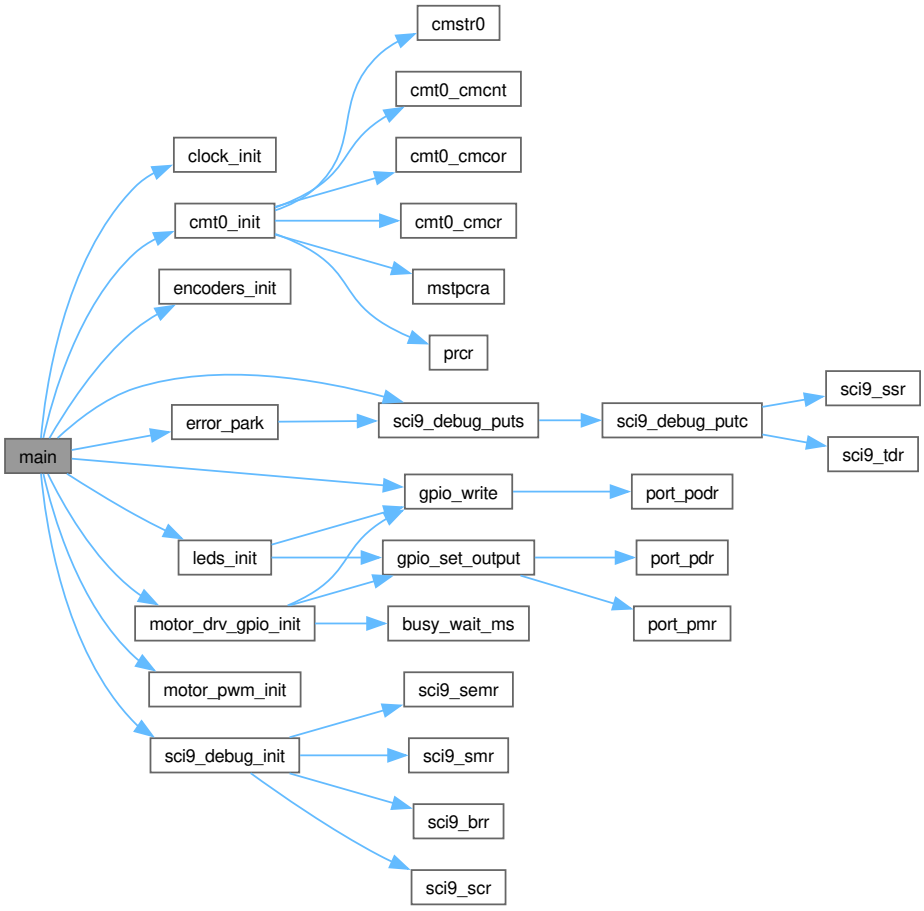
```
00128 {
00129     gpio_set_output(k_port_a, k_bit_led_heartbeat);
00130     gpio_write(k_port_a, k_bit_led_heartbeat, false);
00131
00132     gpio_set_output(k_port_b, k_bit_led_init_done);
00133     gpio_write(k_port_b, k_bit_led_init_done, false);
00134
00135     gpio_set_output(k_port , k_bit_led_motor_run);
00136     gpio_write(k_port , k_bit_led_motor_run, false);
00137
00138     gpio_set_output(k_port , k_bit_led_enc_tick);
00139     gpio_write(k_port , k_bit_led_enc_tick, false);
00140
00141     gpio_set_output(k_port_b, k_bit_led_motor_alive);
00142     gpio_write(k_port_b, k_bit_led_motor_alive, false);
00143 }
```





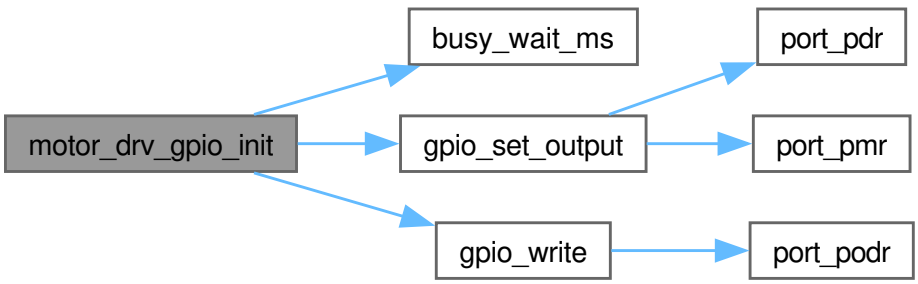
int main (
void)

```
00589 {
00590     clock_init();
00591     leds_init();
00592     sci9_debug_init();
00593
00594     sci9_debug_puts("\n\n");
00595     sci9_debug_puts("motors_rtos: ThreadX + rx_motor + rx_encoder bring-up\n");
00596     sci9_debug_puts("clock=240MHz pclkb=60MHz tick=100Hz\n");
00597
00598     motor_drv_gpio_init();
00599     sci9_debug_puts("drv8263 armed\n");
00600
00601     if (!motor_pwm_init(s_motors)) {
00602         error_park("motor_pwm_init");
00603     }
00604     sci9_debug_puts("motors 0..3 init ok\n");
00605
00606     if (!encoders_init()) {
00607         error_park("encoders_init");
00608     }
00609     sci9_debug_puts("encoders 0..3 init ok\n");
00610
00611     gpio_write(k_port_b, k_bit_led_init_done, true);
00612     sci9_debug_puts("entering scheduler\n");
00613
00614     cmt0_init();
00615     tx_kernel_enter();
00616
00617     /* tx_kernel_enter() never returns; fall-through trap if it somehow does. */
00618     for (;;) {
00619         __asm__ volatile("nop");
00620     }
00621     return 0;
00622 }
```



void motor_drv_gpio_init (void) [static]

```
00217 {
00218     for (uint8_t i = 0; i < k_motor_count; i++) {
00219         gpio_set_output(k_motors[i].drvoff_port, k_motors[i].drvoff_pin);
00220         gpio_write(k_motors[i].drvoff_port, k_motors[i].drvoff_pin, true);
00221     }
00222     for (uint8_t i = 0; i < k_motor_count; i++) {
00223         gpio_set_output(k_motors[i].nsleep_port, k_motors[i].nsleep_pin);
00224         gpio_write(k_motors[i].nsleep_port, k_motors[i].nsleep_pin, true);
00225     }
00226     busy_wait_ms(10);
00227     for (uint8_t i = 0; i < k_motor_count; i++) {
00228         gpio_write(k_motors[i].drvoff_port, k_motors[i].drvoff_pin, false);
00229     }
00230 }
```



bool motor_pwm_init (
rx_motor_handle_t handles[k_motor_count]) [static]

```
00241 {
00242     for (uint8_t i = 0; i < k_motor_count; i++) {
00243         const rx_motor_config_t cfg = {
00244             .channel      = k_motors[i].gptw_channel,
00245             .output_a     = k_gptw_output_a,
00246             .output_b     = k_gptw_output_b,
00247             .pwm_freq_hz  = k_pwm_freq_hz,
00248             .dead_time_ns = k_deadtime_ns,
00249             .invert_pwm   = false,
00250             .port_a_idx   = (uint8_t)rx_port_from_pin(k_motors[i].in2_pin),
00251             .bit_a        = (uint8_t)rx_pin_from_pin(k_motors[i].in2_pin),
00252             .port_b_idx   = (uint8_t)rx_port_from_pin(k_motors[i].in1_pin),
00253             .bit_b        = (uint8_t)rx_pin_from_pin(k_motors[i].in1_pin),
00254         };
00255         if (rx_motor_init(&handles[i], &cfg) != k_rx_ok) {
00256             return false;
00257         }
00258     }
00259     return true;
00260 }
```

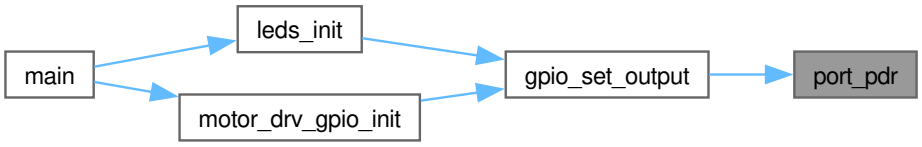
void motor_task_entry (
ULONG arg) [static]

```
00449 {
00450     (void)arg;
00451
00452     g_motor_task_entries = 1U;
00453     gpio_write(k_port_b, k_bit_led_motor_alive, true);
00454
00455     /* Explicit 0 duty on entry for a clean baseline before the sweep. */
00456     for (uint8_t i = 0; i < k_motor_count; i++) {
00457         (void)rx_motor_set_duty(&s_motors[i], 0.0F);
00458     }
00459
00460     uint8_t step = 0U;
00461
00462     for (;;) {
00463         gpio_toggle(k_port , k_bit_led_motor_run);
00464
00465         const int8_t duty = k_sweep_table[step];
00466         s_current_duty = duty;
00467
00468         for (uint8_t i = 0; i < k_motor_count; i++) {
00469             const float signed_duty = (float)(int16_t)duty *
00470                                       (float)(int16_t)k_motors[i].direction_sign;
00471             (void)rx_motor_set_duty(&s_motors[i], signed_duty);
00472         }
00473
00474         step = (uint8_t)((step + 1U) % k_sweep_step_count);
```

```
00475
00476         (void)tx_thread_sleep((ULONG)k_motor_sleep_ticks);
00477     }
00478 }
```

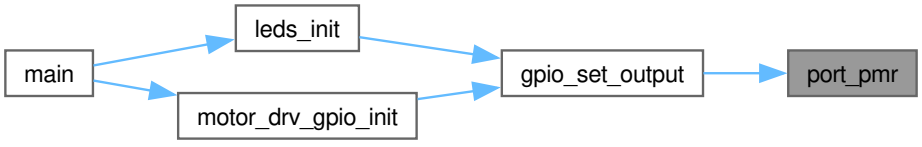
volatile uint8_t * port_pdr (uint8_t port) [inline], [static]

```
00080 {
00081     return &REG8(k_port_pdr_base + port);
00082 }
```



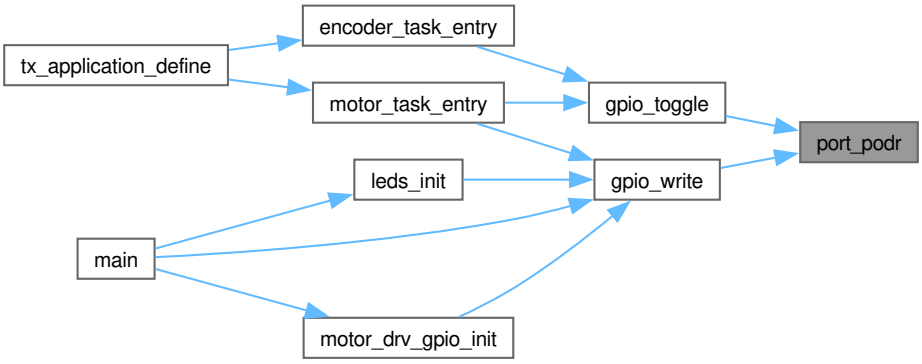
volatile uint8_t * port_pmr (uint8_t port) [inline], [static]

```
00088 {
00089     return &REG8(k_port_pmr_base + port);
00090 }
```



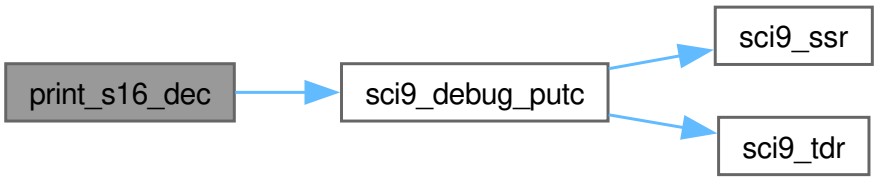
volatile uint8_t * port_podr (
uint8_t port) [inline], [static]

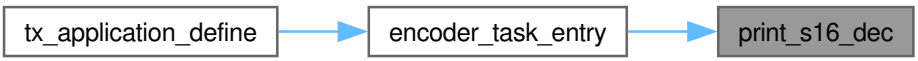
```
00084 {
00085     return &REG8(k_port_podr_base + port);
00086 }
```



void print_s16_dec (
int16_t v) [static]

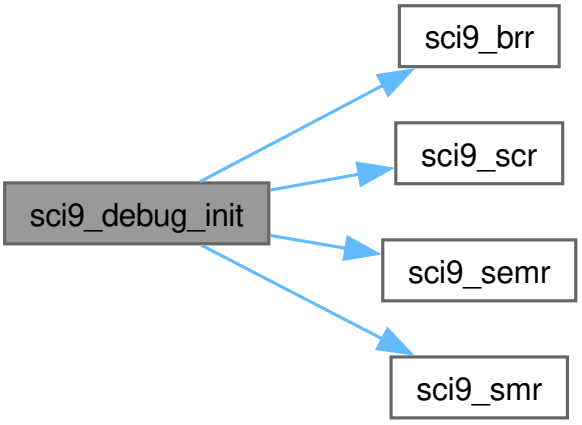
```
00361 {
00362     char buf[8];
00363     uint8_t  i       = 0;
00364     uint32_t u;
00365     bool     negative = v < 0;
00366
00367     u = negative ? (uint32_t)(-(int32_t)v) : (uint32_t)v;
00368     if (u == 0U) {
00369         sci9_debug_putc('0');
00370         return;
00371     }
00372     if (negative) {
00373         sci9_debug_putc('-');
00374     }
00375     while (u != 0U) {
00376         buf[i++] = (char)('0' + (u % 10U));
00377         u /= 10U;
00378     }
00379     while (i-- > 0U) {
00380         sci9_debug_putc(buf[i]);
00381     }
00382 }
```





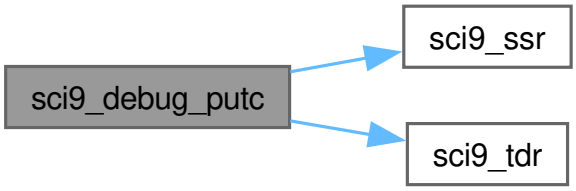
void sci9_debug_init (
void) [extern]

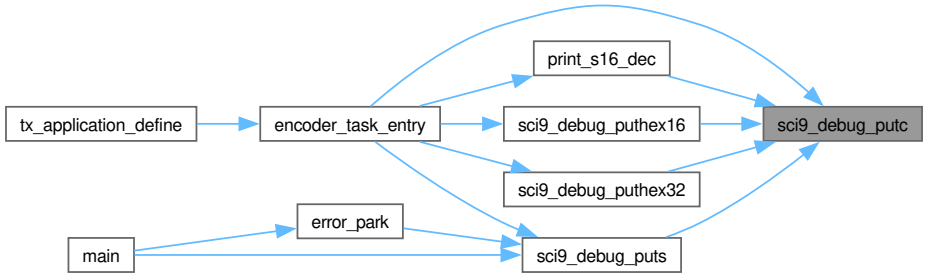
```
00105 {
00106     /* Step 1: release SCI9 from module stop. SCI9 = MSTPCRC bit 26 per
00107      * RX72N HW manual page 410 (NOT MSTPCRB.bit22 -- that's PDC). */
00108     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00109     system_regs()->mstpcrc &= ~(uint32_t)(1UL « (uint32_t)k_mstpc_sci9_bit);
00110     *prcr_reg() = k_rx_prcr_lock;
00111
00112     /* Step 2: configure PB6 (RXD9) and PB7 (TXD9) via MPC.  Use the struct
00113      * accessor so the compiler computes the correct PFS offsets via
00114      * static_assert-checked layout. */
00115     mpc()->pwpr   = k_pwpr_unlock_b0; /* clear B0WI */
00116     mpc()->pwpr   = k_pwpr_pfswe;     /* allow PFS writes */
00117     mpc()->pb6pfs = k_psel_sci9;
00118     mpc()->pb7pfs = k_psel_sci9;
00119     mpc()->pwpr   = k_pwpr_unlock_b0; /* clear PFSWE */
00120     mpc()->pwpr   = k_pwpr_b0wi;      /* re-protect */
00121
00122     /* Step 3: PDR before PMR (production uart.c does this).  PB7 = output
00123      * for TX, PB6 = input for RX.  Then flip PMR to peripheral mode. */
00124     portb()->pdr |= (uint8_t)k_pb7_mask;
00125     portb()->pdr &= (uint8_t) ~k_pb6_mask;
00126     portb()->pmr |= (uint8_t)(k_pb6_mask | k_pb7_mask);
00127
00128     /* Step 4: SCI9 init.  Match the production order:
00129      *   SCR=0 -> SMR -> BRR -> settle -> SCR=TE|RE -> SEMR.
00130      * Don't touch SCMR (reset default 0xF2 is correct). */
00131     *sci9_scr() = k_scr_disabled;
00132     *sci9_smr() = k_smr n1_async;
00133     *sci9_brr() = k_brr ;
00134
00135     /* Wait roughly one bit time before enabling TX/RX. */
00136     for (volatile uint32_t i = 0U; i < (uint32_t)k_brr_settle_loops; i++) {
00137         ___asm___ volatile("nop");
00138     }
00139
00140     *sci9_scr()  = k_scr_txrx_enabled;
00141     *sci9_semr() = k_semr_default;
00142 }
```



void sci9_debug_putc (
char c) [extern]

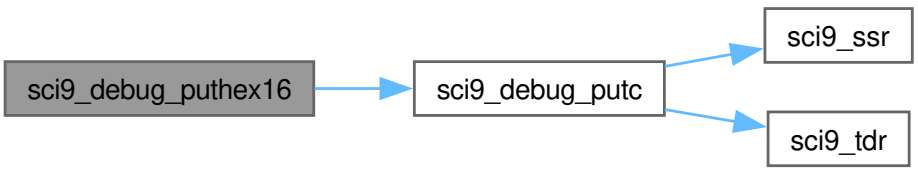
```
00148 {
00149     while ((*sci9_ssr() & (uint8_t)k_ssr_tdre) == 0U) {
00150         /* spin */
00151     }
00152     *sci9_tdr() = (uint8_t)c;
00153 }
```

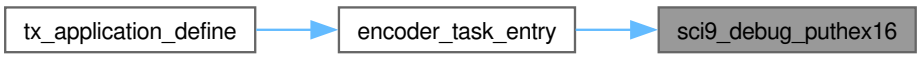




void sci9_debug_puthex16 (
uint16_t v) [extern]

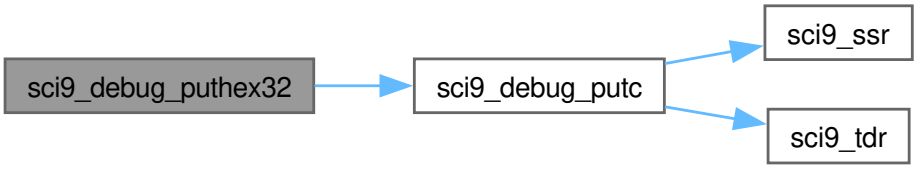
```
00180 {
00181     static const char hex_digits[] = "0123456789ABCDEF";
00182     sci9_debug_putc('0');
00183     sci9_debug_putc('x');
00184     for (int8_t shift = (int8_t)k_hex_shift_init16; shift >= 0;
00185          shift -= (int8_t)k_hex_shift_step) {
00186         sci9_debug_putc(hex_digits[(v >> (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00187     }
00188 }
```

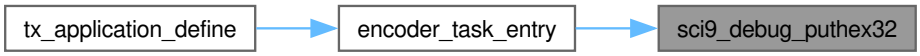




void sci9_debug_puthex32 (
uint32_t v) [extern]

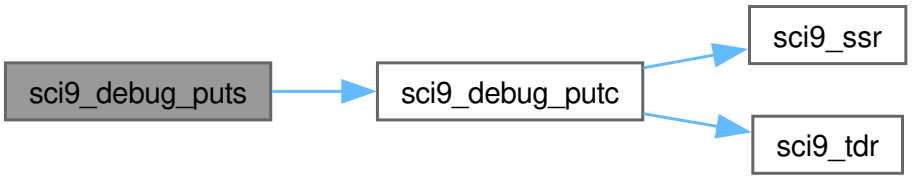
```
00169 {
00170     static const char hex_digits[] = "0123456789ABCDEF";
00171     sci9_debug_putc('0');
00172     sci9_debug_putc('x');
00173     for (int8_t shift = (int8_t)k_hex_shift_init32; shift >= 0;
00174          shift -= (int8_t)k_hex_shift_step) {
00175         sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00176     }
00177 }
```

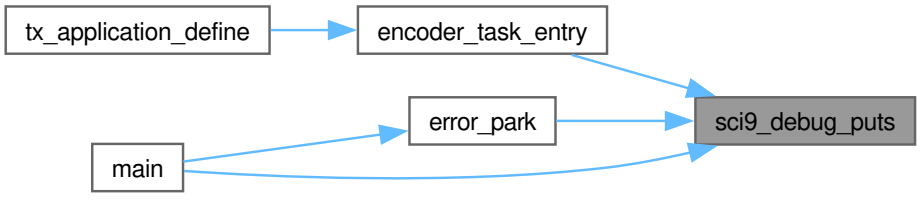




void sci9_debug_puts (
const char * s) [extern]

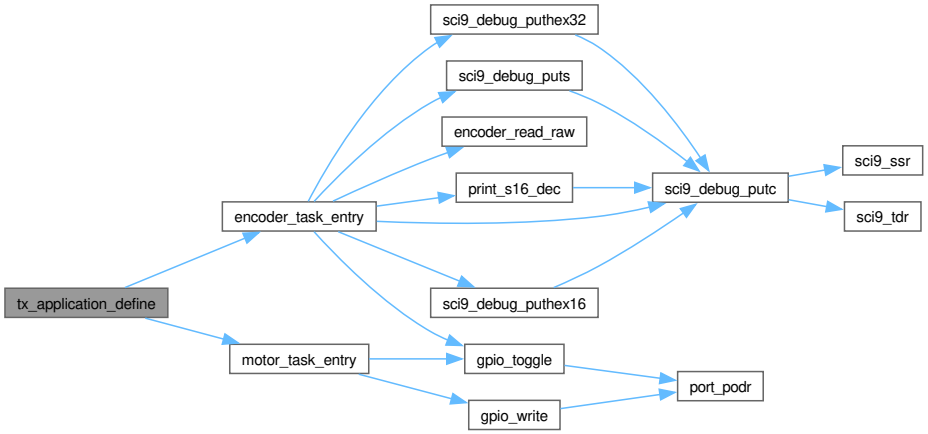
```
00156 {
00157     if (s == (const char *)0) {
00158         return;
00159     }
00160     while (*s != '\0') {
00161         if (*s == '\n') {
00162             sci9_debug_putc('\r');
00163         }
00164         sci9_debug_putc(*s++);
00165     }
00166 }
```





void tx_application_define (
 void * first_unused_memory)

```
00547 {
00548     (void)first_unused_memory;
00549 
00550     g_encoder_create_status = (uint32_t)tx_thread_create(
00551         &s_encoder_tcb, "encoder", encoder_task_entry, 0U,
00552         s_encoder_stack, (ULONG)k_encoder_task_stack_sz,
00553         (UINT)k_encoder_task_priority,
00554         (UINT)k_encoder_task_priority,
00555         TX_NO_TIME_SLICE, TX_AUTO_START);
00556 
00557     g_motor_create_status = (uint32_t)tx_thread_create(
00558         &s_motor_tcb, "motor", motor_task_entry, 0U,
00559         s_motor_stack, (ULONG)k_motor_task_stack_sz,
00560         (UINT)k_motor_task_priority,
00561         (UINT)k_motor_task_priority,
00562         TX_NO_TIME_SLICE, TX_AUTO_START);
00563 }
```



TX_THREAD* _tx_thread_current_ptr [extern]

UINT _tx_thread_preempt_disable [extern]

ULONG _tx_thread_system_state [extern]

volatile uint32_t g_cmt0_isr_count [extern]

volatile uint32_t g_encoder_create_status = 0xFFFFFFFFU [static]

volatile uint32_t g_motor_create_status = 0xFFFFFFFFU [static]

volatile uint32_t g_motor_task_entries = 0U [static]

volatile uint32_t g_motor_task_loops = 0U [static]

volatile uint32_t g_motor_task_loops_after = 0U [static]

const motor_wiring_t k_motors[k_motor_count] [static]

= {

```
    {.gptw_channel = k_gptw_channel , .in2_pin = k_rx_p2 , .in1_pin = k_rx_p1 ,
     .drvoff_port = k_port , .drvoff_pin = 1, .nsleep_port = k_port , .nsleep_pin = 0,
     .is_tpu = false, .timer_channel = 1, .direction_sign = -1},

    {.gptw_channel = k_gptw_channel , .in2_pin = k_rx_p2 , .in1_pin = k_rx_pc ,
     .drvoff_port = k_port , .drvoff_pin = 3, .nsleep_port = k_port , .nsleep_pin = 2,
     .is_tpu = false, .timer_channel = 2, .direction_sign = +1},

    {.gptw_channel = k_gptw_channel , .in2_pin = k_rx_pe , .in1_pin = k_rx_p8 ,
     .drvoff_port = k_port_e, .drvoff_pin = 0, .nsleep_port = k_port , .nsleep_pin = 4,
     .is_tpu = true,  .timer_channel = 1, .direction_sign = +1},

    {.gptw_channel = k_gptw_channel , .in2_pin = k_rx_pe , .in1_pin = k_rx_pc ,
     .drvoff_port = k_port_e, .drvoff_pin = 2, .nsleep_port = k_port_e, .nsleep_pin = 1,
     .is_tpu = true,  .timer_channel = 2, .direction_sign = -1},
}
```

```
00176                                       {
00177     /* M0 = FL, wired backwards */
00178     {.gptw_channel = k_gptw_channel , .in2_pin = k_rx_p2 , .in1_pin = k_rx_p1 ,
00179      .drvoff_port = k_port , .drvoff_pin = 1, .nsleep_port = k_port , .nsleep_pin = 0,
00180      .is_tpu = false, .timer_channel = 1, .direction_sign = -1},
00181     /* M1 = FR */
00182     {.gptw_channel = k_gptw_channel , .in2_pin = k_rx_p2 , .in1_pin = k_rx_pc ,
00183      .drvoff_port = k_port , .drvoff_pin = 3, .nsleep_port = k_port , .nsleep_pin = 2,
00184      .is_tpu = false, .timer_channel = 2, .direction_sign = +1},
00185     /* M2 = BR */
00186     {.gptw_channel = k_gptw_channel , .in2_pin = k_rx_pe , .in1_pin = k_rx_p8 ,
00187      .drvoff_port = k_port_e, .drvoff_pin = 0, .nsleep_port = k_port , .nsleep_pin = 4,
00188      .is_tpu = true,  .timer_channel = 1, .direction_sign = +1},
00189     /* M3 = BL, wired backwards */
00190     {.gptw_channel = k_gptw_channel , .in2_pin = k_rx_pe , .in1_pin = k_rx_pc ,
00191      .drvoff_port = k_port_e, .drvoff_pin = 2, .nsleep_port = k_port_e, .nsleep_pin = 1,
00192      .is_tpu = true,  .timer_channel = 2, .direction_sign = -1},
00193 };
```

const int8_t k_sweep_table[k_sweep_step_count] [static]

= {
-100, -90, -80, -70, -60, -50, -40, -30, -20, -10,
0,
+10, +20, +30, +40, +50, +60, +70, +80, +90, +100,
}

00392 {
00393 -100, -90, -80, -70, -60, -50, -40, -30, -20, -10,
00394 0,
00395 +10, +20, +30, +40, +50, +60, +70, +80, +90, +100,
00396 };

volatile int8_t s_current_duty = 0 [static]

uint8_t s_encoder_stack[k_encoder_task_stack_sz] [static]

TX_THREAD s_encoder_tcb [static]

uint8_t s_motor_stack[k_motor_task_stack_sz] [static]

TX_THREAD s_motor_tcb [static]

rx_motor_handle_t s_motors[k_motor_count] [static]

```
00001 /**
00002  * @file main.c
00003  * @brief motors_rtos -- motors + encoders under Azure RTOS ThreadX.
00004  *
00005  * @details
00006  * Two-task ThreadX bring-up of libs/rx_motor + libs/rx_encoder, proving they
00007  * coexist with the scheduler.  Mirrors encoder_test's open-loop spin + count
00008  * logic but splits the work across two cooperating tasks:
00009  *
00010  *   motor_task    (priority 10) -- every 500 ms, advances a duty-sweep state
00011  *                                  machine.  All 4 motors take the same
00012  *                                  duty, with per-motor direction_sign applied
00013  *                                  so FL/BL spin the same wheel direction as
00014  *                                  FR/BR despite the reversed wiring.
00015  *
00016  *   encoder_task  (priority  9) -- every 100 ms, reads all 4 encoder TCNTs,
00017  *                                  prints a one-line CSV over SCI9 (115200,
00018  *                                  /dev/ttyACM0 on the host), and toggles the
00019  *                                  PA7/D9 heartbeat LED.  Higher priority
00020  *                                  than motor_task so the console stays
00021  *                                  responsive even when motor_task is awake.
00022  *
00023  * Boot sequence (main):
00024  *   1. clock_init()         -- PLL 240 MHz / PCLKA 120 / PCLKB 60
00025  *   2. sci9_debug_init()    -- console up; banner printed immediately
00026  *   3. motor_drv_gpio_init  -- DRVOFF high, nSLEEP high, tWAKE 10 ms, DRVOFF low
00027  *   4. motor_pwm_init       -- rx_motor_init x4 at 20 kHz, 1 us deadtime
00028  *   5. encoders_init        -- MTU1/MTU2/TPU1/TPU2 phase counting mode 1
00029  *   6. cmt0_init            -- 100 Hz ThreadX tick + setpsw i
00030  *   7. tx_kernel_enter      -- hands off forever
00031  *
00032  * Per memory project_motor_pwm_hal_bugs.md: the production rx_mpc / rx_gptw
00033  * stack has latent issues writing PFS for MTCLK/TCLK encoder pins, so the
00034  * encoders_init() routine here mirrors encoder_test's direct-register workaround
00035  * rather than calling rx_mpc_set_mtu_encoder() / rx_mpc_set_tpu_encoder().
00036  *
00037  * SPDX-License-Identifier: MIT
00038  * @copyright Copyright (c) 2026 Locked Inc.
00039  */
00040
00041 #include <stdint.h>
00042 #include <stdbool.h>
00043 #include <stddef.h>
00044
00045 #include "tx_api.h"
00046
00047 #include "rx_motor.h"
00048 #include "rx_gptw.h"
00049 #include "rx_mpc.h"
00050 #include "rx_port_constants.h"
00051 #include "rx_mtu_encoder.h"
00052 #include "rx_encoder_tpu.h"
00053 #include "rx72n_mtu_regs.h"
00054 #include "rx72n_tpu_regs.h"
00055 #include "rx72n_system_regs.h"
00056 #include "rx72n_icu_regs.h"
00057 #include "rx_register_protection.h"
00058
00059 /* Symbols defined in sibling files */
00060 extern void clock_init(void);
00061 extern void cmt0_init(void);
00062 extern void sci9_debug_init(void);
00063 extern void sci9_debug_putc(char c);
00064 extern void sci9_debug_puts(const char *s);
00065 extern void sci9_debug_puthex16(uint16_t v);
00066 extern void sci9_debug_puthex32(uint32_t v);
00067
00068 /*
      ============================================================================
00069  * Direct port-register access helpers (same pattern as encoder_test).
00070  *
      ============================================================================
      */
00071 #define REG8(a) (*(volatile uint8_t *)(uintptr_t)(a))
00072
00073 typedef enum : uintptr_t {
00074     k_port_pdr_base  = 0x0008C000U,
00075     k_port_podr_base = 0x0008C020U,
00076     k_port_pmr_base  = 0x0008C060U,
00077 } port_reg_base_t;
00078
00079 static inline volatile uint8_t *port_pdr(uint8_t port)
00080 {
00081     return &REG8(k_port_pdr_base + port);
00082 }
00083 static inline volatile uint8_t *port_podr(uint8_t port)
```

```
00084 {
00085     return &REG8(k_port__podr__base + port);
00086 }
00087 static inline volatile uint8_t *port__pmr(uint8_t port)
00088 {
00089     return &REG8(k_port__pmr__base + port);
00090 }
00091
00092 static inline void gpio__set__output(uint8_t port, uint8_t pin)
00093 {
00094     *port__pmr(port) &= (uint8_t) ~(1U « pin);
00095     *port__pdr(port) |= (uint8_t)(1U « pin);
00096 }
00097 static inline void gpio__write(uint8_t port, uint8_t pin, bool high)
00098 {
00099     if (high) {
00100         *port__podr(port) |= (uint8_t)(1U « pin);
00101     } else {
00102         *port__podr(port) &= (uint8_t) ~(1U « pin);
00103     }
00104 }
00105 static inline void gpio__toggle(uint8_t port, uint8_t pin)
00106 {
00107     *port__podr(port) ^= (uint8_t)(1U « pin);
00108 }
00109
00110 /*
    ================================================================================
00111  * Status LED map (matches encoder_test, matches memory project_star_pcb_leds.md)
00112  *
    ================================================================================
    */
00113 typedef enum : uint8_t {
00114     k_port    = 7,
00115     k_port__a   = 10,
00116     k_port__b   = 11,
00117     k_port__e   = 14,
00118     k_port    = 6,
00119
00120     k_bit__led__heartbeat = 7, /**< PA7 -- D9, toggled 10 Hz by encoder_task */
00121     k_bit__led__init__done = 0, /**< PB0 -- D10, solid once main() finishes init */
00122     k_bit__led__motor__run = 1, /**< P71 -- D11, toggles on every motor_task step */
00123     k_bit__led__enc__tick  = 2, /**< P72 -- D12, toggles per encoder sample */
00124     k_bit__led__motor__alive = 1, /**< PB1 -- D13, proves motor_task reached loop body */
00125 } status__led__pins__t;
00126
00127 static void leds__init(void)
00128 {
00129     gpio__set__output(k_port__a, k_bit__led__heartbeat);
00130     gpio__write(k_port__a, k_bit__led__heartbeat, false);
00131
00132     gpio__set__output(k_port__b, k_bit__led__init__done);
00133     gpio__write(k_port__b, k_bit__led__init__done, false);
00134
00135     gpio__set__output(k_port , k_bit__led__motor__run);
00136     gpio__write(k_port , k_bit__led__motor__run, false);
00137
00138     gpio__set__output(k_port , k_bit__led__enc__tick);
00139     gpio__write(k_port , k_bit__led__enc__tick, false);
00140
00141     gpio__set__output(k_port__b, k_bit__led__motor__alive);
00142     gpio__write(k_port__b, k_bit__led__motor__alive, false);
00143 }
00144
00145 /*
    ================================================================================
00146  * Per-motor wiring (mirrors encoder_test / src/inc/hardware_config.h).
00147  *
00148  * | Idx | Wheel | GPTW | IN1  | IN2  | DRVOFF | nSLEEP | Encoder |
00149  * |-----|-------|------|------|------|--------|--------|---------|
00150  * |  0  | FL    | 0    | P17  | P23  | P61    | P60    | MTU1    |
00151  * |  1  | FR    | 1    | PC3  | P22  | P63    | P62    | MTU2    |
00152  * |  2  | BR    | 2    | P86  | PE3  | PE0    | P64    | TPU1    |
00153  * |  3  | BL    | 3    | PC6  | PE7  | PE2    | PE1    | TPU2    |
00154  *
00155  * direction_sign compensates for the physical wiring: M0 (FL) and M3 (BL)
00156  * are reversed on this PCB revision, so +50% duty for "forward" applies as
00157  * -50% to those channels.
00158  *
    ================================================================================
    */
00159 typedef struct {
00160     rx__gptw__channel__t gptw__channel;
00161     rx__port__pin__t     in2__pin;
00162     rx__port__pin__t     in1__pin;
00163     uint8__t           drvoff__port;
00164     uint8__t           drvoff__pin;
```

```
00165     uint8_t           nsleep_port;
00166     uint8_t           nsleep_pin;
00167     bool              is_tpu;
00168     uint8_t           timer_channel;
00169     int8_t            direction_sign;
00170 } motor_wiring_t;
00171
00172 typedef enum : uint8_t {
00173     k_motor_count = 4,
00174 } motor_const_t;
00175
00176 static const motor_wiring_t k_motors[k_motor_count] = {
00177     /* M0 = FL, wired backwards */
00178     {.gptw_channel = k_gptw_channel , .in2_pin = k_rx_p2 , .in1_pin = k_rx_p1 ,
00179      .drvoff_port = k_port , .drvoff_pin = 1, .nsleep_port = k_port , .nsleep_pin = 0,
00180      .is_tpu = false, .timer_channel = 1, .direction_sign = -1},
00181     /* M1 = FR */
00182     {.gptw_channel = k_gptw_channel , .in2_pin = k_rx_p2 , .in1_pin = k_rx_pc ,
00183      .drvoff_port = k_port , .drvoff_pin = 3, .nsleep_port = k_port , .nsleep_pin = 2,
00184      .is_tpu = false, .timer_channel = 2, .direction_sign = +1},
00185     /* M2 = BR */
00186     {.gptw_channel = k_gptw_channel , .in2_pin = k_rx_pe , .in1_pin = k_rx_p8 ,
00187      .drvoff_port = k_port_e, .drvoff_pin = 0, .nsleep_port = k_port , .nsleep_pin = 4,
00188      .is_tpu = true,  .timer_channel = 1, .direction_sign = +1},
00189     /* M3 = BL, wired backwards */
00190     {.gptw_channel = k_gptw_channel , .in2_pin = k_rx_pe , .in1_pin = k_rx_pc ,
00191      .drvoff_port = k_port_e, .drvoff_pin = 2, .nsleep_port = k_port_e, .nsleep_pin = 1,
00192      .is_tpu = true,  .timer_channel = 2, .direction_sign = -1},
00193 };
00194
00195 /*
    ============================================================================
00196  * Bounded busy-wait (~1 ms per unit at 240 MHz ICLK, -O1, volatile loop).
00197  * Only used in main() before tx_kernel_enter() for the DRV8263 tWAKE window;
00198  * after that, every delay goes through tx_thread_sleep().
00199  *
    ============================================================================
    */
00200 typedef enum : uint32_t {
00201     k_iters_per_ms = 40000U,
00202 } delay_const_t;
00203
00204 static void busy_wait_ms(uint32_t ms)
00205 {
00206     for (uint32_t m = 0; m < ms; m++) {
00207         for (volatile uint32_t i = 0; i < k_iters_per_ms; i++) {
00208             ___asm___ volatile("nop");
00209         }
00210     }
00211 }
00212
00213 /*
    ============================================================================
00214  * DRV8263H power-up: DRVOFF high first, nSLEEP high, 10 ms tWAKE, DRVOFF low.
00215  *
    ============================================================================
    */
00216 static void motor_drv_gpio_init(void)
00217 {
00218     for (uint8_t i = 0; i < k_motor_count; i++) {
00219         gpio_set_output(k_motors[i].drvoff_port, k_motors[i].drvoff_pin);
00220         gpio_write(k_motors[i].drvoff_port, k_motors[i].drvoff_pin, true);
00221     }
00222     for (uint8_t i = 0; i < k_motor_count; i++) {
00223         gpio_set_output(k_motors[i].nsleep_port, k_motors[i].nsleep_pin);
00224         gpio_write(k_motors[i].nsleep_port, k_motors[i].nsleep_pin, true);
00225     }
00226     busy_wait_ms(10);
00227     for (uint8_t i = 0; i < k_motor_count; i++) {
00228         gpio_write(k_motors[i].drvoff_port, k_motors[i].drvoff_pin, false);
00229     }
00230 }
00231
00232 /*
    ============================================================================
00233  * GPTW PWM init via rx_motor lib (same config as motor_spin_test).
00234  *
    ============================================================================
    */
00235 typedef enum : uint32_t {
00236     k_pwm_freq_hz  = 20000U,
00237     k_deadtime_ns  = 1000U,
00238 } pwm_const_t;
00239
00240 static bool motor_pwm_init(rx_motor_handle_t handles[k_motor_count])
00241 {
00242     for (uint8_t i = 0; i < k_motor_count; i++) {
```

```
00243         const rx_motor_config_t cfg = {
00244             .channel      = k_motors[i].gptw_channel,
00245             .output_a     = k_gptw_output_a,
00246             .output_b     = k_gptw_output_b,
00247             .pwm_freq_hz  = k_pwm_freq_hz,
00248             .dead_time_ns = k_deadtime_ns,
00249             .invert_pwm   = false,
00250             .port_a_idx   = (uint8_t)rx_port_from_pin(k_motors[i].in2_pin),
00251             .bit_a        = (uint8_t)rx_pin_from_pin(k_motors[i].in2_pin),
00252             .port_b_idx   = (uint8_t)rx_port_from_pin(k_motors[i].in1_pin),
00253             .bit_b        = (uint8_t)rx_pin_from_pin(k_motors[i].in1_pin),
00254         };
00255         if (rx_motor_init(&handles[i], &cfg) != k_rx_ok) {
00256             return false;
00257         }
00258     }
00259     return true;
00260 }
00261
00262 /*
    ============================================================================
00263  * Encoder init -- direct-register, bypassing rx_mpc / rx_mtu_start which
00264  * don't actually write PFS or TSTRA on this HAL revision.  Copied verbatim
00265  * from the known-good encoder_test/main.c encoders_init() routine.
00266  *
    ============================================================================
    */
00267 #define REG8_AT(a)  (*(volatile uint8_t  *)(uintptr_t)(a))
00268 #define REG16_AT(a) (*(volatile uint16_t *)(uintptr_t)(a))
00269
00270 static bool encoders_init(void)
00271 {
00272     /* Step 1: release MTU (MSTPA9) + TPU (MSTPA13) module clocks. */
00273     *prcr_reg() = k_rx_prcr_unlock_all;
00274     system_regs()->mstpcra &= ~(uint32_t)((1UL << 9) | (uint32_t)k_tpu_mstpcra_mstpa13);
00275     *prcr_reg() = k_rx_prcr_lock;
00276
00277     /* Step 2 + 3: MPC pin sequence PMR=0, write PFS, PMR=1 for every
00278      * encoder phase input. */
00279     volatile uint8_t *pwpr  = (volatile uint8_t *)0x0008C11FU;
00280     volatile uint8_t *p2pmr = (volatile uint8_t *)0x0008C062U;
00281     volatile uint8_t *papmr = (volatile uint8_t *)0x0008C06AU;
00282     volatile uint8_t *pbpmr = (volatile uint8_t *)0x0008C06BU;
00283     volatile uint8_t *pcpmr = (volatile uint8_t *)0x0008C06CU;
00284
00285     *p2pmr &= (uint8_t)~(uint8_t)((1U << 4) | (1U << 5));
00286     *papmr &= (uint8_t)~(uint8_t)((1U << 1) | (1U << 3));
00287     *pcpmr &= (uint8_t)~(uint8_t)((1U << 0) | (1U << 2) | (1U << 5));
00288     *pbpmr &= (uint8_t)~(uint8_t)(1U << 3);
00289
00290     *pwpr = 0x00U;
00291     *pwpr = 0x40U;
00292     REG8_AT(0x0008C140U + 2U * 8U + 4U)  = 0x02U; /* P24 MTCLKA */
00293     REG8_AT(0x0008C140U + 2U * 8U + 5U)  = 0x02U; /* P25 MTCLKB */
00294     REG8_AT(0x0008C140U + 10U * 8U + 1U) = 0x02U; /* PA1 MTCLKC */
00295     REG8_AT(0x0008C140U + 12U * 8U + 5U) = 0x02U; /* PC5 MTCLKD */
00296     REG8_AT(0x0008C140U + 12U * 8U + 2U) = 0x03U; /* PC2 TCLKA  */
00297     REG8_AT(0x0008C140U + 10U * 8U + 3U) = 0x04U; /* PA3 TCLKB  */
00298     REG8_AT(0x0008C140U + 12U * 8U + 0U) = 0x03U; /* PC0 TCLKC  */
00299     REG8_AT(0x0008C140U + 11U * 8U + 3U) = 0x04U; /* PB3 TCLKD  */
00300     *pwpr = 0x00U;
00301     *pwpr = 0x80U;
00302
00303     *p2pmr |= (uint8_t)((1U << 4) | (1U << 5));
00304     *papmr |= (uint8_t)((1U << 1) | (1U << 3));
00305     *pcpmr |= (uint8_t)((1U << 0) | (1U << 2) | (1U << 5));
00306     *pbpmr |= (uint8_t)(1U << 3);
00307
00308     /* Phase counting mode 1 on MTU1 + MTU2 */
00309     volatile rx_mtu_channel_regs_t *m1 = mtu1();
00310     m1->tcr = 0;  m1->tmdr = 0x04U;
00311     m1->tiorh = 0; m1->tiorl = 0;
00312     m1->tier = 0;  m1->tsr = 0;  m1->tcnt = 0;
00313     volatile rx_mtu_channel_regs_t *m2 = mtu2();
00314     m2->tcr = 0;  m2->tmdr = 0x04U;
00315     m2->tiorh = 0; m2->tiorl = 0;
00316     m2->tier = 0;  m2->tsr = 0;  m2->tcnt = 0;
00317
00318     /* Phase counting mode 1 on TPU1 + TPU2 */
00319     volatile rx_tpu_regs_t *t1 = tpu1();
00320     t1->tcr = 0;  t1->tmdr = 0x04U;  t1->tcnt = 0;
00321     volatile rx_tpu_regs_t *t2 = tpu2();
00322     t2->tcr = 0;  t2->tmdr = 0x04U;  t2->tcnt = 0;
00323
00324     /* Clear TPU noise filters from any prior-run leftover state */
00325     tpu_control()->nfcr[1] = 0;
00326     tpu_control()->nfcr[2] = 0;
```

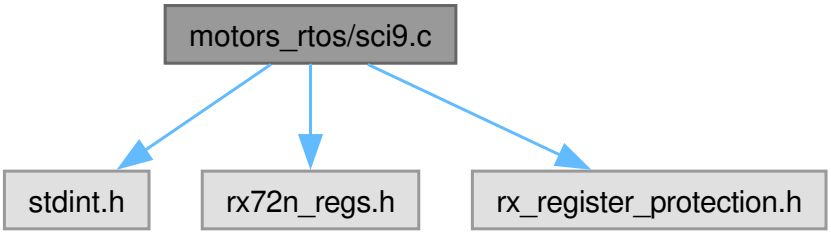
```
00327
00328     /* Start all four counters */
00329     mtu_tstra()->tstr   |= (uint8_t)(k_mtu_tstr_cst1 | k_mtu_tstr_cst2);
00330     tpu_control()->tstr |= (uint8_t)(k_tpu_tstr_cst1 | k_tpu_tstr_cst2);
00331
00332     return true;
00333 }
00334
00335 static uint16_t encoder_read_raw(uint8_t motor_idx)
00336 {
00337     /* IMPORTANT -- indices 2 and 3 are INTENTIONALLY SWAPPED relative to the
00338      * k_motors[] table's `.timer_channel` field and the previous "M2->TPU1,
00339      * M3->TPU2" comment.  Cross-check against motor_spin_test IPROPI on
00340      * 2026-04-21 showed M2 (BR) drawing ~0 current at every duty (dead
00341      * motor, hardware issue) while the encoder at TPU1 was counting fast
00342      * and the encoder at TPU2 was pinned at 0.  The only consistent reading
00343      * is that the physical encoder harness wires BR's encoder to TPU2
00344      * (TCLKC/TCLKD on PC0/PB3) and BL's encoder to TPU1 (TCLKA/TCLKB on
00345      * PC2/PA3) -- the opposite of what the k_motors[] comment claims.
00346      * Swapping the indices here makes m2 track BR and m3 track BL so the
00347      * CSV column matches the motor driver index in motor_spin_test. */
00348     static const uintptr_t k_tcnt_addr[k_motor_count] = {
00349         0x000C1386U,        /* MTU1 -- M0 (FL) */
00350         0x000C1406U,        /* MTU2 -- M1 (FR) */
00351         0x00088130U + 6U,   /* TPU2 -- M2 (BR): harness wires BR -> TPU2 */
00352         0x00088120U + 6U,   /* TPU1 -- M3 (BL): harness wires BL -> TPU1 */
00353     };
00354     return REG16_AT(k_tcnt_addr[motor_idx]);
00355 }
00356
00357 /*
    ============================================================================
00358  * Console helpers -- thin wrappers around sci9_debug_* for readability.
00359  *
    ============================================================================
    */
00360 static void print_s16_dec(int16_t v)
00361 {
00362     char buf[8];
00363     uint8_t  i        = 0;
00364     uint32_t u;
00365     bool     negative = v < 0;
00366
00367     u = negative ? (uint32_t)(-(int32_t)v) : (uint32_t)v;
00368     if (u == 0U) {
00369         sci9_debug_putc('0');
00370         return;
00371     }
00372     if (negative) {
00373         sci9_debug_putc('-');
00374     }
00375     while (u != 0U) {
00376         buf[i++] = (char)('0' + (u % 10U));
00377         u /= 10U;
00378     }
00379     while (i-- > 0U) {
00380         sci9_debug_putc(buf[i]);
00381     }
00382 }
00383
00384 /*
    ============================================================================
00385  * Duty sweep state machine -- 21 duty steps from -100 to +100 in 10% steps,
00386  * reversed at the endpoints.  motor_task advances one step per wakeup.
00387  *
    ============================================================================
    */
00388 typedef enum : uint8_t {
00389     k_sweep_step_count = 21U, /**< -100, -90, ..., 0, ..., +100 */
00390 } sweep_const_t;
00391
00392 static const int8_t k_sweep_table[k_sweep_step_count] = {
00393     -100, -90, -80, -70, -60, -50, -40, -30, -20, -10,
00394        0,
00395      +10, +20, +30, +40, +50, +60, +70, +80, +90, +100,
00396 };
00397
00398 /*
    ============================================================================
00399  * ThreadX objects
00400  *
    ============================================================================
    */
00401 typedef enum : uint16_t {
00402     k_motor_task_stack_sz   = 2048U,
00403     k_encoder_task_stack_sz = 2048U,
00404 } rtos_stack_t;
```

```
00405
00406 typedef enum : uint8_t {
00407     k_encoder_task_priority = 9U,  /**< higher priority: prints CSV, keeps console responsive */
00408     k_motor_task_priority   = 10U, /**< lower priority: sweeps duty, preempted by encoder */
00409     k_motor_sleep_ticks     = 50U, /**< 50 * 10 ms = 500 ms */
00410     k_encoder_sleep_ticks   = 10U, /**< 10 * 10 ms = 100 ms */
00411 } rtos_prio_t;
00412
00413 static TX_THREAD s_motor_tcb;
00414 static TX_THREAD s_encoder_tcb;
00415 static uint8_t   s_motor_stack[k_motor_task_stack_sz];
00416 static uint8_t   s_encoder_stack[k_encoder_task_stack_sz];
00417
00418 /* Motor handles initialised in main() before the scheduler starts. */
00419 static rx_motor_handle_t s_motors[k_motor_count];
00420
00421 /* Most recent motor duty -- written by motor_task, read by encoder_task for
00422  * CSV logging.  Signed volatile byte is atomic on RX so no mutex needed. */
00423 static volatile int8_t s_current_duty = 0;
00424
00425 /* Diagnostic counters -- help identify *where* motor_task is making progress.
00426  * All three are volatile so the compiler can't optimise them away. */
00427 static volatile uint32_t g_motor_task_entries     = 0U; /**< bumped once on task entry */
00428 static volatile uint32_t g_motor_task_loops       = 0U; /**< bumped each iteration BEFORE set_duty */
00429 static volatile uint32_t g_motor_task_loops_after = 0U; /**< bumped each iteration AFTER  set_duty */
00430 static volatile uint32_t g_motor_create_status    = 0xFFFFFFFFU; /**< tx_thread_create return for motor_task */
00431 static volatile uint32_t g_encoder_create_status  = 0xFFFFFFFFU; /**< tx_thread_create return for encoder_task */
00432
00433 extern volatile uint32_t g_cmt0_isr_count; /**< from cmt0.c -- verifies CMT0 tick ISR is firing */
00434
00435 /* ThreadX kernel internals -- used by Option 1/1b diagnostic in encoder_task.
00436  * tx_thread_sleep.c:89-118 has four TX_CALLER_ERROR (0x13) return paths:
00437  *   :89   thread_ptr == NULL          -> observe via cur
00438  *   :99   system_state != 0           -> observe via ss  (ISR wrapping issue)
00439  *   :111  thread_ptr == timer thread  -> observe via cur
00440  *   :132  preempt_disable != 0        -> observe via pd  (already ruled out: pd=0) */
00441 extern UINT       _tx_thread_preempt_disable;
00442 extern ULONG      _tx_thread_system_state;
00443 extern TX_THREAD *_tx_thread_current_ptr;
00444
00445 /*
    ================================================================================
00446  * motor_task -- step the duty sweep across all 4 motors every 500 ms.
00447  *
    ================================================================================
    */
00448 static void motor_task_entry(ULONG arg)
00449 {
00450     (void)arg;
00451
00452     g_motor_task_entries = 1U;
00453     gpio_write(k_port_b, k_bit_led_motor_alive, true);
00454
00455     /* Explicit 0 duty on entry for a clean baseline before the sweep. */
00456     for (uint8_t i = 0; i < k_motor_count; i++) {
00457         (void)rx_motor_set_duty(&s_motors[i], 0.0F);
00458     }
00459
00460     uint8_t step = 0U;
00461
00462     for (;;) {
00463         gpio_toggle(k_port , k_bit_led_motor_run);
00464
00465         const int8_t duty = k_sweep_table[step];
00466         s_current_duty = duty;
00467
00468         for (uint8_t i = 0; i < k_motor_count; i++) {
00469             const float signed_duty = (float)(int16_t)duty *
00470                                       (float)(int16_t)k_motors[i].direction_sign;
00471             (void)rx_motor_set_duty(&s_motors[i], signed_duty);
00472         }
00473
00474         step = (uint8_t)((step + 1U) % k_sweep_step_count);
00475
00476         (void)tx_thread_sleep((ULONG)k_motor_sleep_ticks);
00477     }
00478 }
00479
00480 /*
    ================================================================================
00481  * encoder_task -- 10 Hz CSV dump + heartbeat LED.
00482  *
00483  * CSV header (printed once at task start): t_ms,duty,m0,m1,m2,m3
00484  * Each subsequent line: uptime,duty,cnt0,cnt1,cnt2,cnt3
00485  *
    ================================================================================
    */
```

```
00486 static void encoder_task_entry(ULONG arg)
00487 {
00488     (void)arg;
00489
00490     sci9_debug_puts("t_ms,cmt0_cnt,mot_state,m_ent,duty,m0,m1,m2,m3\n");
00491
00492     uint32_t t_ms = 0U;
00493
00494     for (;;) {
00495         gpio_toggle(k_port_a, k_bit_led_heartbeat);
00496         gpio_toggle(k_port , k_bit_led_enc_tick);
00497
00498         const uint16_t c0 = encoder_read_raw(0U);
00499         const uint16_t c1 = encoder_read_raw(1U);
00500         const uint16_t c2 = encoder_read_raw(2U);
00501         const uint16_t c3 = encoder_read_raw(3U);
00502
00503
00504         print_s16_dec((int16_t)(t_ms & 0x7FFFU));
00505         sci9_debug_putc(',');
00506         print_s16_dec((int16_t)(g_cmt0_isr_count & 0x7FFFU));
00507         sci9_debug_putc(',');
00508         print_s16_dec((int16_t)s_motor_tcb.tx_thread_state);
00509         sci9_debug_putc(',');
00510         print_s16_dec((int16_t)g_motor_task_entries);
00511         sci9_debug_putc(',');
00512         print_s16_dec((int16_t)s_current_duty);
00513         sci9_debug_putc(',');
00514         print_s16_dec((int16_t)c0);
00515         sci9_debug_putc(',');
00516         print_s16_dec((int16_t)c1);
00517         sci9_debug_putc(',');
00518         print_s16_dec((int16_t)c2);
00519         sci9_debug_putc(',');
00520         print_s16_dec((int16_t)c3);
00521         sci9_debug_putc('\n');
00522
00523         /* Option 1b diagnostic: capture all TX_CALLER_ERROR discriminators
00524          * BEFORE sleep plus its return value AFTER.  ss and cur distinguish
00525          * the three remaining TX_CALLER_ERROR paths in tx_thread_sleep.c. */
00526         const UINT        pd  = _tx_thread_preempt_disable;
00527         const ULONG       ss  = _tx_thread_system_state;
00528         const TX_THREAD  *cur = _tx_thread_current_ptr;
00529         const UINT        ret = tx_thread_sleep((ULONG)k_encoder_sleep_ticks);
00530         sci9_debug_puts("pd=");
00531         print_s16_dec((int16_t)pd);
00532         sci9_debug_puts(" ss=");
00533         sci9_debug_puthex32((uint32_t)ss);
00534         sci9_debug_puts(" cur=");
00535         sci9_debug_puthex32((uint32_t)(uintptr_t)cur);
00536         sci9_debug_puts(" ret=");
00537         sci9_debug_puthex16((uint16_t)ret);
00538         sci9_debug_putc('\n');
00539         t_ms += 100U;
00540     }
00541 }
00542
00543 /*
    ============================================================================
00544  * ThreadX application-define -- create both tasks (hardware already init'd).
00545  *
    ============================================================================
    */
00546 void tx_application_define(void *first_unused_memory)
00547 {
00548     (void)first_unused_memory;
00549
00550     g_encoder_create_status = (uint32_t)tx_thread_create(
00551         &s_encoder_tcb, "encoder", encoder_task_entry, 0U,
00552         s_encoder_stack, (ULONG)k_encoder_task_stack_sz,
00553         (UINT)k_encoder_task_priority,
00554         (UINT)k_encoder_task_priority,
00555         TX_NO_TIME_SLICE, TX_AUTO_START);
00556
00557     g_motor_create_status = (uint32_t)tx_thread_create(
00558         &s_motor_tcb, "motor", motor_task_entry, 0U,
00559         s_motor_stack, (ULONG)k_motor_task_stack_sz,
00560         (UINT)k_motor_task_priority,
00561         (UINT)k_motor_task_priority,
00562         TX_NO_TIME_SLICE, TX_AUTO_START);
00563 }
00564
00565 /*
    ============================================================================
00566  * Fatal error trap -- park all four motors and halt with the FAIL LED lit.
00567  *
    ============================================================================
```

```
     */
00568 static void error_park(const char *msg)
00569 {
00570     sci9_debug_puts("FATAL ");
00571     sci9_debug_puts(msg);
00572     sci9_debug_puts("\n");
00573
00574     /* Best-effort motor safe: attempt a 0-duty write (may fail if rx_motor
00575      * isn't initialised, but that's OK since GPTW would be idle in that case). */
00576     for (uint8_t i = 0; i < k_motor_count; i++) {
00577         (void)rx_motor_set_duty(&s_motors[i], 0.0F);
00578     }
00579
00580     for (;;) {
00581         __asm__ volatile("nop");
00582     }
00583 }
00584
00585 /*
     ============================================================================
00586  * main() -- runs ONCE on reset, init everything, hand off to ThreadX forever.
00587  *
     ============================================================================
     */
00588 int main(void)
00589 {
00590     clock_init();
00591     leds_init();
00592     sci9_debug_init();
00593
00594     sci9_debug_puts("\n\n");
00595     sci9_debug_puts("motors_rtos: ThreadX + rx_motor + rx_encoder bring-up\n");
00596     sci9_debug_puts("clock=240MHz pclkb=60MHz tick=100Hz\n");
00597
00598     motor_drv_gpio_init();
00599     sci9_debug_puts("drv8263 armed\n");
00600
00601     if (!motor_pwm_init(s_motors)) {
00602         error_park("motor_pwm_init");
00603     }
00604     sci9_debug_puts("motors 0..3 init ok\n");
00605
00606     if (!encoders_init()) {
00607         error_park("encoders_init");
00608     }
00609     sci9_debug_puts("encoders 0..3 init ok\n");
00610
00611     gpio_write(k_port_b, k_bit_led_init_done, true);
00612     sci9_debug_puts("entering scheduler\n");
00613
00614     cmt0_init();
00615     tx_kernel_enter();
00616
00617     /* tx_kernel_enter() never returns; fall-through trap if it somehow does. */
00618     for (;;) {
00619         __asm__ volatile("nop");
00620     }
00621     return 0;
00622 }
```

```
#include <stdint.h>
  #include "rx72n_regs.h"
  #include "rx_register_protection.h"
```

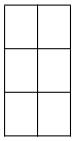


<<<<

*
*
*
*
*
*
*

*

enum hex_shifts_t : uint8_t

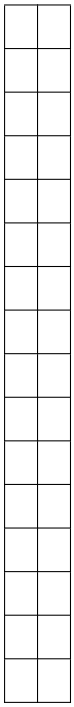


```
00062             : uint8_t {
00063     k_hex_shift_init32 = 28U,
00064     k_hex_shift_init16 = 12U,
00065     k_hex_shift_step   = 4U,
00066 } hex_shifts_t;
```

enum sci9_addrs_t : uintptr_t

```
00035             : uintptr_t {
00036     k_sci9_base = 0x000D0020U,
00037 } sci9_addrs_t;
```

enum sci9_cfg_t : uint8_t



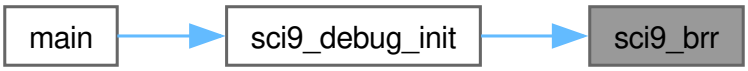
```
00039             : uint8_t {
00040     k_psel_sci9        = 0x0AU, /**< SCI9 RXD/TXD function */
00041     k_pb6_mask          = (uint8_t)(1U << 6U),
00042     k_pb7_mask          = (uint8_t)(1U << 7U),
00043     k_mstpc_sci9_bit    = 26U, /* MSTPCRC.MSTPC26 per RX72N HW manual page 410 */
00044     k_pwpr_unlock_b0    = 0x00U,
00045     k_pwpr_pfswe        = 0x40U,
00046     k_pwpr_b0wi         = 0x80U,
00047     k_brr         = 31U, /* SCI9 uses PCLKA=120 MHz (SCI7-11 group); BRR=31 @ CKS=0 ABCS=0 -> ~117 kbaud
        (1.7% over 115200) */
00048     k_smr_n1_async      = 0x00U,
00049     k_scr_te_only       = 0x20U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=0 */
00050     k_scr_txrx_enabled = 0x30U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=1 */
00051     k_scr_disabled      = 0x00U,
00052     k_ssr_tdre          = 0x80U,
00053     k_scmr_default      = 0xF2U,
00054     k_semr_default      = 0x00U,
00055     k_hex_nibble_mask   = 0x0FU,
00056 } sci9_cfg_t;
```

enum sci9_settle_t : uint16_t

```
00058             : uint16_t {
00059     k_brr_settle_loops = 1000U, /**< Match production uart.c (~8.68 us @ 115200) */
00060 } sci9_settle_t;
```

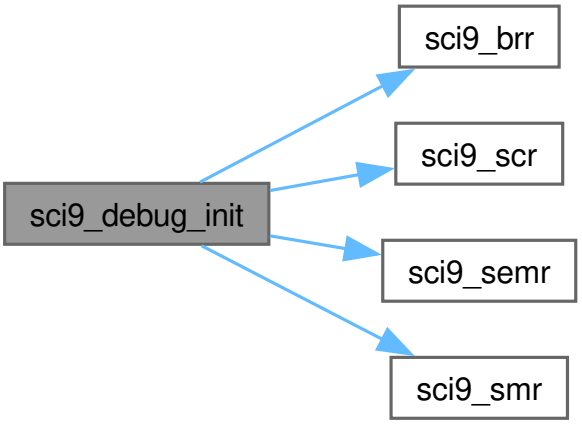
volatile uint8_t * sci9_brr (
void) [inline], [static]

```
00077 {
00078     return (volatile uint8_t *)(k_sci9_base + 1U);
00079 }
```



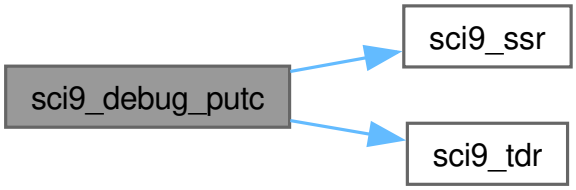

```
void sci9_debug_init (
                    void )
```

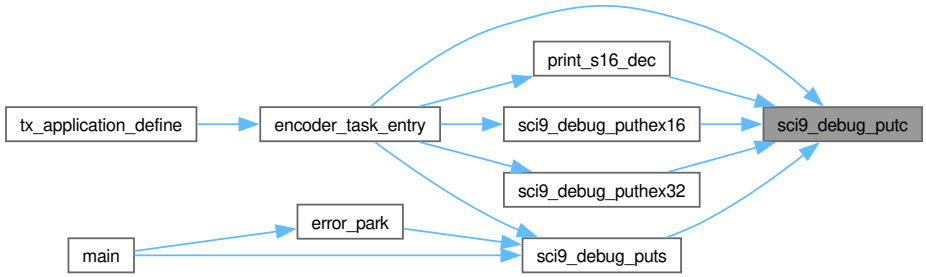
```
00105 {
00106     /* Step 1: release SCI9 from module stop. SCI9 = MSTPCRC bit 26 per
00107      * RX72N HW manual page 410 (NOT MSTPCRB.bit22 -- that's PDC). */
00108     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00109     system_regs()->mstpcrc &= ~(uint32_t)(1UL « (uint32_t)k_mstpc_sci9_bit);
00110     *prcr_reg() = k_rx_prcr_lock;
00111
00112     /* Step 2: configure PB6 (RXD9) and PB7 (TXD9) via MPC.  Use the struct
00113      * accessor so the compiler computes the correct PFS offsets via
00114      * static_assert-checked layout. */
00115     mpc()->pwpr   = k_pwpr_unlock_b0; /* clear B0WI */
00116     mpc()->pwpr   = k_pwpr_pfswe;     /* allow PFS writes */
00117     mpc()->pb6pfs = k_psel_sci9;
00118     mpc()->pb7pfs = k_psel_sci9;
00119     mpc()->pwpr   = k_pwpr_unlock_b0; /* clear PFSWE */
00120     mpc()->pwpr   = k_pwpr_b0wi;      /* re-protect */
00121
00122     /* Step 3: PDR before PMR (production uart.c does this).  PB7 = output
00123      * for TX, PB6 = input for RX.  Then flip PMR to peripheral mode. */
00124     portb()->pdr |= (uint8_t)k_pb7_mask;
00125     portb()->pdr &= (uint8_t) ~k_pb6_mask;
00126     portb()->pmr |= (uint8_t)(k_pb6_mask | k_pb7_mask);
00127
00128     /* Step 4: SCI9 init.  Match the production order:
00129      *   SCR=0 -> SMR -> BRR -> settle -> SCR=TE|RE -> SEMR.
00130      * Don't touch SCMR (reset default 0xF2 is correct). */
00131     *sci9_scr() = k_scr_disabled;
00132     *sci9_smr() = k_smr n1_async;
00133     *sci9_brr() = k_brr ;
00134
00135     /* Wait roughly one bit time before enabling TX/RX. */
00136     for (volatile uint32_t i = 0U; i < (uint32_t)k_brr_settle_loops; i++) {
00137         __asm__ volatile("nop");
00138     }
00139
00140     *sci9_scr()  = k_scr_txrx_enabled;
00141     *sci9_semr() = k_semr_default;
00142 }
```



void sci9_debug_putc (
char c)

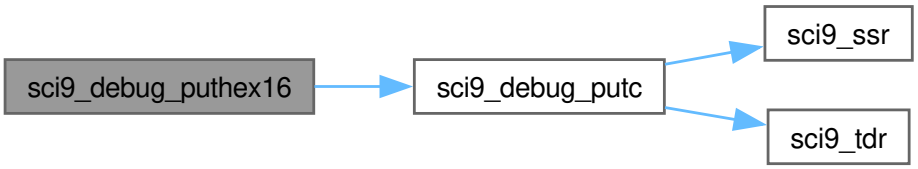
```
00148 {
00149     while ((*sci9_ssr() & (uint8_t)k_ssr_tdre) == 0U) {
00150         /* spin */
00151     }
00152     *sci9_tdr() = (uint8_t)c;
00153 }
```

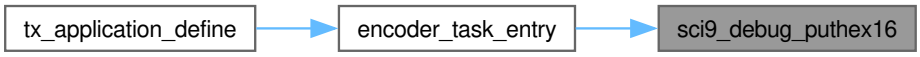




void sci9_debug_puthex16 (
uint16_t v)

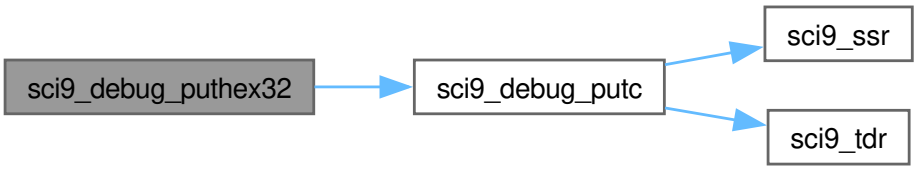
```
00180 {
00181     static const char hex_digits[] = "0123456789ABCDEF";
00182     sci9_debug_putc('0');
00183     sci9_debug_putc('x');
00184     for (int8_t shift = (int8_t)k_hex_shift_init16; shift >= 0;
00185          shift -= (int8_t)k_hex_shift_step) {
00186         sci9_debug_putc(hex_digits[(v >> (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00187     }
00188 }
```

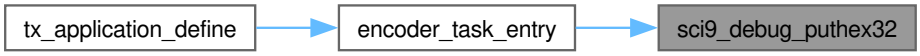




void sci9_debug_puthex32 (
uint32_t v)

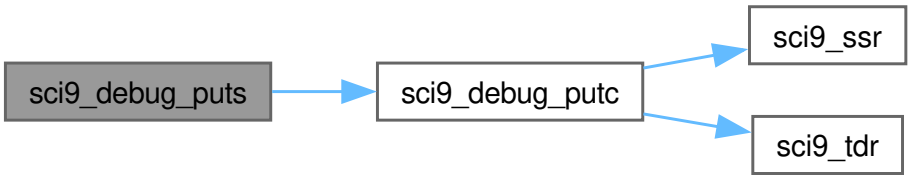
```
00169 {
00170     static const char hex_digits[] = "0123456789ABCDEF";
00171     sci9_debug_putc('0');
00172     sci9_debug_putc('x');
00173     for (int8_t shift = (int8_t)k_hex_shift_init32; shift >= 0;
00174          shift -= (int8_t)k_hex_shift_step) {
00175         sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00176     }
00177 }
```

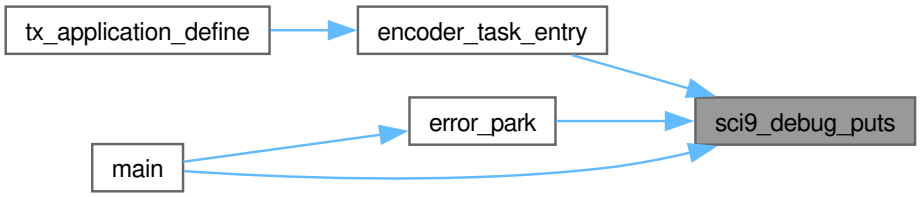




void sci9_debug_puts (
const char * s)

```
00156 {
00157     if (s == (const char *)0) {
00158         return;
00159     }
00160     while (*s != '\0') {
00161         if (*s == '\n') {
00162             sci9_debug_putc('\r');
00163         }
00164         sci9_debug_putc(*s++);
00165     }
00166 }
```



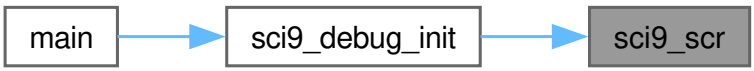


volatile uint8_t * sci9_scmr (
void) [inline], [static]

```
00093 {
00094     return (volatile uint8_t *)(k_sci9_base + 6U);
00095 }
```

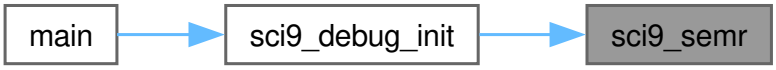
volatile uint8_t * sci9_scr (
void) [inline], [static]

```
00081 {
00082     return (volatile uint8_t *)(k_sci9_base + 2U);
00083 }
```



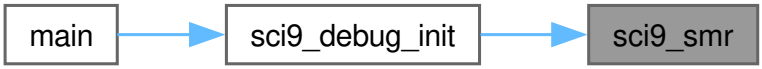
volatile uint8_t * sci9_semr (
void) [inline], [static]

```
00097 {
00098     return (volatile uint8_t *)(k_sci9_base + 7U);
00099 }
```



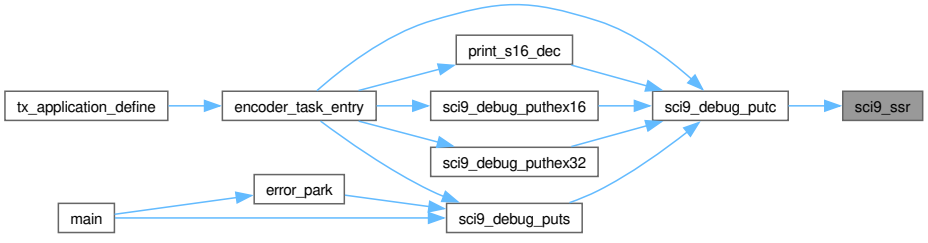
volatile uint8_t * sci9_smr (
void) [inline], [static]

```
00073 {
00074     return (volatile uint8_t *)(k_sci9_base + 0U);
00075 }
```



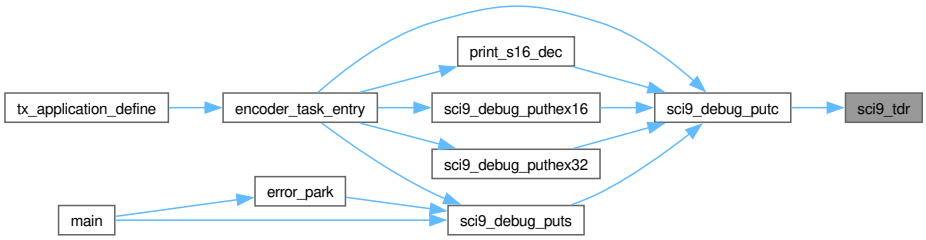
volatile uint8_t * sci9_ssr (
void) [inline], [static]

```
00089 {
00090     return (volatile uint8_t *)(k_sci9_base + 4U);
00091 }
```



volatile uint8_t * sci9_tdr (
void) [inline], [static]

```
00085 {
00086     return (volatile uint8_t *)(k_sci9_base + 3U);
00087 }
```

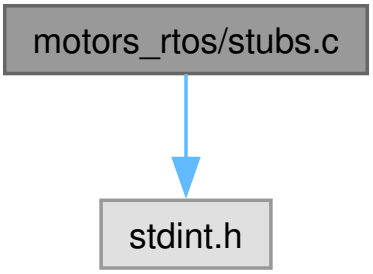



```
00001 /**
00002  * @file sci9_debug.c
00003  * @brief Minimal polled-mode SCI9 (TXD9 = PB7, RXD9 = PB6) debug UART for usb_test.
00004  *
00005  * @details
00006  * The STAR board routes SCI9 to the on-board CY7C65213 USB-to-UART bridge
00007  * which appears as /dev/ttyACM0 on the host (or COMx on Windows).  This
00008  * file provides just enough init + putc/puts to emit ASCII status from
00009  * the firmware so we can debug the USB CDC bring-up without needing the
00010  * USB device to actually enumerate.
00011  *
00012  * Uses the rx_hal struct accessors (mpc(), portb(), system_regs(), prcr_reg())
00013  * so the addresses are all compiler-verified via static_assert.
00014  *
00015  * Baud = 115200. SCI9 is in the SCI7-11 group that uses PCLKA (120 MHz),
00016  * not PCLKB. At CKS=0, ABCS=0:
00017  *   BRR = (PCLKA / (32 * baud)) - 1 = 120000000 / 3686400 - 1 = 31.55 -> 31
00018  *   Actual baud = PCLKA / (32 * (BRR+1)) = 117187 Hz (+1.7% vs 115200, in tolerance).
00019  *
00020  * SPDX-License-Identifier: MIT
00021  * @copyright Copyright (c) 2026 Locked Inc.
00022  */
00023
00024 #include <stdint.h>
00025
00026 #include "rx72n_regs.h"
00027 #include "rx_register_protection.h"
00028
00029 /*
      ================================================================================
00030  * SCI9 register access via raw addresses (rx72n_sci_regs.h doesn't expose
00031  * the per-channel struct as cleanly as we'd like for a quick polled driver).
00032  * The base 0x000D0020 is taken from the verified header.
00033  * Register offsets per RX72N HW manual: SMR=0, BRR=1, SCR=2, TDR=3, SSR=4.
00034  *
      ================================================================================
      */
00035 typedef enum : uintptr_t {
00036     k_sci9_base = 0x000D0020U,
00037 } sci9_addrs_t;
00038
00039 typedef enum : uint8_t {
00040     k_psel_sci9        = 0x0AU, /**< SCI9 RXD/TXD function */
00041     k_pb6_mask         = (uint8_t)(1U << 6U),
00042     k_pb7_mask         = (uint8_t)(1U << 7U),
00043     k_mstpc_sci9_bit   = 26U, /* MSTPCRC.MSTPC26 per RX72N HW manual page 410 */
00044     k_pwpr_unlock_b0   = 0x00U,
00045     k_pwpr_pfswe       = 0x40U,
00046     k_pwpr_b0wi        = 0x80U,
00047     k_brr        = 31U, /* SCI9 uses PCLKA=120 MHz (SCI7-11 group); BRR=31 @ CKS=0 ABCS=0 -> ~117 kbaud
      (1.7% over 115200) */
```

```
00048     k_smr_n1_async    = 0x00U,
00049     k_scr_te_only     = 0x20U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=0 */
00050     k_scr_txrx_enabled = 0x30U, /**< CKE=00, MPIE=0, RIE=0, TE=1, RE=1 */
00051     k_scr_disabled    = 0x00U,
00052     k_ssr_tdre        = 0x80U,
00053     k_scmr_default    = 0xF2U,
00054     k_semr_default    = 0x00U,
00055     k_hex_nibble_mask  = 0x0FU,
00056 } sci9_cfg_t;
00057
00058 typedef enum : uint16_t {
00059     k_brr_settle_loops = 1000U, /**< Match production uart.c (~8.68 us @ 115200) */
00060 } sci9_settle_t;
00061
00062 typedef enum : uint8_t {
00063     k_hex_shift_init32 = 28U,
00064     k_hex_shift_init16 = 12U,
00065     k_hex_shift_step   = 4U,
00066 } hex_shifts_t;
00067
00068 /*
      ============================================================================
00069  * SCI9 register accessors (raw addresses since the header doesn't give us a
00070  * per-channel struct for the SCIi/SCIj quirks).
00071  *
      ============================================================================
      */
00072 static inline volatile uint8_t *sci9_smr(void)
00073 {
00074     return (volatile uint8_t *)(k_sci9_base + 0U);
00075 }
00076 static inline volatile uint8_t *sci9_brr(void)
00077 {
00078     return (volatile uint8_t *)(k_sci9_base + 1U);
00079 }
00080 static inline volatile uint8_t *sci9_scr(void)
00081 {
00082     return (volatile uint8_t *)(k_sci9_base + 2U);
00083 }
00084 static inline volatile uint8_t *sci9_tdr(void)
00085 {
00086     return (volatile uint8_t *)(k_sci9_base + 3U);
00087 }
00088 static inline volatile uint8_t *sci9_ssr(void)
00089 {
00090     return (volatile uint8_t *)(k_sci9_base + 4U);
00091 }
00092 static inline volatile uint8_t *sci9_scmr(void)
00093 {
00094     return (volatile uint8_t *)(k_sci9_base + 6U);
00095 }
00096 static inline volatile uint8_t *sci9_semr(void)
00097 {
00098     return (volatile uint8_t *)(k_sci9_base + 7U);
00099 }
00100
00101 /*
      ============================================================================
00102  * Initialise SCI9 for 115200 8N1 polled output on PB7 (TXD9).
00103  *
      ============================================================================
      */
00104 void sci9_debug_init(void)
00105 {
00106     /* Step 1: release SCI9 from module stop. SCI9 = MSTPCRC bit 26 per
00107      * RX72N HW manual page 410 (NOT MSTPCRB.bit22 -- that's PDC). */
00108     *prcr_reg() = k_rx_prcr_unlock_prc0_prc1;
00109     system_regs()->mstpcrc &= ~(uint32_t)(1UL << (uint32_t)k_mstpc_sci9_bit);
00110     *prcr_reg() = k_rx_prcr_lock;
00111
00112     /* Step 2: configure PB6 (RXD9) and PB7 (TXD9) via MPC.  Use the struct
00113      * accessor so the compiler computes the correct PFS offsets via
00114      * static_assert-checked layout. */
00115     mpc()->pwpr   = k_pwpr_unlock_b0; /* clear B0WI */
00116     mpc()->pwpr   = k_pwpr_pfswe;     /* allow PFS writes */
00117     mpc()->pb6pfs = k_psel_sci9;
00118     mpc()->pb7pfs = k_psel_sci9;
00119     mpc()->pwpr   = k_pwpr_unlock_b0; /* clear PFSWE */
00120     mpc()->pwpr   = k_pwpr_b0wi;      /* re-protect */
00121
00122     /* Step 3: PDR before PMR (production uart.c does this).  PB7 = output
00123      * for TX, PB6 = input for RX.  Then flip PMR to peripheral mode. */
00124     portb()->pdr |= (uint8_t)k_pb7_mask;
00125     portb()->pdr &= (uint8_t) ~k_pb6_mask;
00126     portb()->pmr |= (uint8_t)(k_pb6_mask | k_pb7_mask);
00127
00128     /* Step 4: SCI9 init.  Match the production order:
```

```
00129      *    SCR=0 -> SMR -> BRR -> settle -> SCR=TE|RE -> SEMR.
00130      * Don't touch SCMR (reset default 0xF2 is correct). */
00131     *sci9_scr() = k_scr_disabled;
00132     *sci9_smr() = k_smr n1_async;
00133     *sci9_brr() = k_brr ;
00134
00135     /* Wait roughly one bit time before enabling TX/RX. */
00136     for (volatile uint32_t i = 0U; i < (uint32_t)k_brr_settle_loops; i++) {
00137         __asm__ volatile("nop");
00138     }
00139
00140     *sci9_scr()  = k_scr_txrx_enabled;
00141     *sci9_semr() = k_semr_default;
00142 }
00143
00144 /*
     ============================================================================
00145  * Polled write: spin until TDRE then drop byte into TDR.
00146  *
     ============================================================================
     */
00147 void sci9_debug_putc(char c)
00148 {
00149     while ((*sci9_ssr() & (uint8_t)k_ssr_tdre) == 0U) {
00150         /* spin */
00151     }
00152     *sci9_tdr() = (uint8_t)c;
00153 }
00154
00155 void sci9_debug_puts(const char *s)
00156 {
00157     if (s == (const char *)0) {
00158         return;
00159     }
00160     while (*s != '\0') {
00161         if (*s == '\n') {
00162             sci9_debug_putc('\r');
00163         }
00164         sci9_debug_putc(*s++);
00165     }
00166 }
00167
00168 void sci9_debug_puthex32(uint32_t v)
00169 {
00170     static const char hex_digits[] = "0123456789ABCDEF";
00171     sci9_debug_putc('0');
00172     sci9_debug_putc('x');
00173     for (int8_t shift = (int8_t)k_hex_shift_init32; shift >= 0;
00174          shift -= (int8_t)k_hex_shift_step) {
00175         sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00176     }
00177 }
00178
00179 void sci9_debug_puthex16(uint16_t v)
00180 {
00181     static const char hex_digits[] = "0123456789ABCDEF";
00182     sci9_debug_putc('0');
00183     sci9_debug_putc('x');
00184     for (int8_t shift = (int8_t)k_hex_shift_init16; shift >= 0;
00185          shift -= (int8_t)k_hex_shift_step) {
00186         sci9_debug_putc(hex_digits[(v » (uint8_t)shift) & (uint8_t)k_hex_nibble_mask]);
00187     }
00188 }
```

#include <stdint.h>



*

*

void rx_log_uart_putc (
    char c)

00020 { (void)c; }

void rx_log_uart_puthex (
    uint32_t v,
    uint8_t d)

00024 { (void)v; (void)d; }

void rx_log_uart_putint (
    int32_t v)

00022 { (void)v; }

void rx_log_uart_puts (
    const char * s)

00021 { (void)s; }

void rx_log_uart_putuint (
    uint32_t v)

00023 { (void)v; }

```
void uart_debug_putc (
                    char c)


 00027 { (void)c; }

void uart_debug_puthex (
                    uint32_t v,
                    uint8_t d)


 00031 { (void)v; (void)d; }

void uart_debug_putint (
                    int32_t v)


 00029 { (void)v; }

void uart_debug_puts (
                    const char * s)


 00028 { (void)s; }

void uart_debug_putuint (
                    uint32_t v)


 00030 { (void)v; }
```

```
 00001 /**
 00002  * @file stubs.c
 00003  * @brief No-op log/UART stubs for the motor spin test.
 00004  *
 00005  * @details
 00006  * The production rx_motor / rx_gptw / rx_mpc libraries call rx_log_error()
 00007  * and RX_ASSERT() macros that ultimately reduce to a handful of UART log
 00008  * primitives. This bench test does not bring up SCI9 or the framed UART
 00009  * ring buffer, so we satisfy the linker with no-op stubs. RX_ASSERT
 00010  * failures still halt (the static-inline internal_rx_fatal_error in
 00011  * rx_check.h disables interrupts and spins).
 00012  *
 00013  * SPDX-License-Identifier: MIT
 00014  * @copyright Copyright (c) 2026 Locked Inc.
 00015  */
 00016
 00017 #include <stdint.h>
 00018
 00019 /* rx_log_uart family (selected when RX_IS_SIMULATOR == 0). */
 00020 void rx_log_uart_putc(char c)              { (void)c; }
 00021 void rx_log_uart_puts(const char *s)       { (void)s; }
 00022 void rx_log_uart_putint(int32_t v)         { (void)v; }
 00023 void rx_log_uart_putuint(uint32_t v)       { (void)v; }
 00024 void rx_log_uart_puthex(uint32_t v, uint8_t d) { (void)v; (void)d; }
 00025
 00026 /* uart_debug family (called unconditionally by internal_rx_fatal_error). */
 00027 void uart_debug_putc(char c)               { (void)c; }
 00028 void uart_debug_puts(const char *s)        { (void)s; }
 00029 void uart_debug_putint(int32_t v)          { (void)v; }
 00030 void uart_debug_putuint(uint32_t v)        { (void)v; }
 00031 void uart_debug_puthex(uint32_t v, uint8_t d) { (void)v; (void)d; }
```